

Team Number:	apmcm24200636
Problem Chosen:	B

2024APMCM summary sheet

This problem pertains to optimization planning, where single-objective and multi-objective programming models were established to determine the optimal shape and size parameters of the equipment using various suitable optimization algorithms.

For Problem 1, a high-performance air-conditioner optimization model was developed, simulating temperature variations in summer and winter under different influencing factors, and designing the optimal shape and size. Initially, the Navier-Stokes equation was used to describe airflow. Subsequently, a single-objective programming model was established with objectives of temperature uniformity, energy efficiency, and noise control, with volume as a constraint. The model was solved using a combination of random search and finite difference methods, identifying wind speed, quantity, and spatial position as key factors, with the optimal shape and size being a cube of 0.5m. Simulations of temperature regulation in summer and winter showed better performance in summer.

For Problem 2, a high-performance air purifier optimization model was established to design the optimal shape and size. The Advection-Diffusion equation was used to simulate molecular diffusion. A single-objective programming model was then established with maximum purification efficiency as the objective and purifier volume as the constraint, solved using the gradient descent algorithm. The optimal shape and size were determined as a rectangular prism of $0.82\text{m} \times 0.92\text{m} \times 1.18\text{m}$. Robustness testing showed minimal impact on purification under uneven pollutant distribution, indicating high model robustness.

For Problem 3, a high-performance humidifier optimization model was developed to determine the optimal shape and size. Fick's law was introduced to establish a water vapor diffusion equation. A single-objective programming model was then established with optimal humidification effect as the objective and volume limitation as the constraint, solved using the BFGS optimization algorithm. The optimal shape and size were found to be a rectangular prism of $1.26\text{m} \times 0.2\text{m} \times 0.5\text{m}$. Sensitivity analysis showed minimal change in maximum humidification efficiency with increasing power, indicating good model stability.

For Problem 4, a Pareto optimization model was established to determine the optimal shape and size of a three-in-one device. Building on the previous models, this model comprehensively considered energy efficiency, human comfort, purification effect, and humidification effect maximization as objectives, establishing a multi-objective programming model. The model was solved using the SLSQP algorithm, yielding an optimal shape and size for the three-in-one device as a rectangular prism of $1.24\text{m} \times 1.24\text{m} \times 2.15\text{m}$, with defined height ranges for each functional module within the device.

Keywords: Navier-Stokes equation; Advection-diffusion equation; Fick's law; various optimization algorithms

Directory

1	Introduction.....	3
1.1	Problem Background.....	3
1.2	Question Restatement.....	3
2	problem analysis	4
2.1	analysis of problem 1	4
2.2	analysis of problem 2	4
2.3	analysis of problem 3	5
2.4	analysis of problem 4	5
3	Model assumptions	5
4	Symbol Description	6
5	Our Work.....	6
6	establishment and solution of the model.....	7
6.1	Model establishment and solution of problem 1	7
6.2	Model establishment and solution of problem 2	16
6.3	Model establishment and solution of problem 3	20
6.4	Model establishment and solution of problem 4	24
7	evaluation and generalization of the model	26
7.1	evaluation of the model.....	26
7.2	generalization of the model	27
	References.....	28
	Appendix.....	29

1 Introduction

1.1 Problem Background

At present, as economic development enters a high-quality stage, people's demand for quality of life is also increasing. As far as environmental electrical equipment is concerned, environmental conditioning devices combining air conditioners, humidifiers and air purifiers are gradually emerging. The device fully integrates the three devices and replenishes each other. The air conditioner will dry the indoor air while adjusting the temperature, and the humidifier can replenish moisture in time to properly adjust the indoor temperature. And when the indoor air circulation caused by the operation of the air conditioner, the air purifier can filter harmful pollutants in the air, purify the circulating air, and improve the indoor air quality. In addition, the three-in-one device is equipped with electrical appliances that control 3 devices at the same time, reducing the power supply and connecting wires of multiple devices, and reducing the potential safety hazards of excessive lines.

Through variable refrigerant flow, the air conditioner integrates the compressor, condenser and evaporator to form an artificial entity system with the function of providing a comfortable indoor environment. The working principle of the air purifier is to use high-voltage electricity to convert air molecules into positive ions and free electrons, so that they are adsorbed and deposited on the deposition plate. When the air enters the photocatalyst catalytic decomposition area, the harmful substances in the air are decomposed into negative oxygen ions and water. At this point, the air purification work is completed. The original reason of the humidifier is that alternating current is transformed into direct current through transformer step-down rectification, and then the direct current is transformed into high-frequency alternating current through an oscillation circuit and added to ultrasonic atomization. If external force is applied to it to deform, it will output corresponding voltage and current. On the contrary, applying alternating voltage to it will produce corresponding deformation, that is, vibration. In this way, the ceramic oscillator is driven by the high frequency voltage to produce ultrasonic oscillation, so that the water is atomized, and then the water mist is sent out by the fan to achieve the effect of humidifying the air.

1.2 Question Restatement

Access to relevant information shows that the design of integrated air conditioners, humidifiers and air purifiers is closely related to aerodynamics. The overall shape, position, quantity, direction, angle, wind speed and air volume of the air inlet and outlet will affect the effectiveness of the three integrated environmental equipment.

As the manager of the air conditioner appearance optimization team, the mathematical model of the three-in-one air conditioner appearance optimization is established according to the specific topic, and the air conditioner appearance is designed for a room with an approximate cuboid space area of $5\text{m} \times 8\text{m} \times 3\text{m}$ and is about to be renovated.

For the problem 1, considering the different shapes of air conditioning, the

influence factors of air conditioning position, air inlet position, quantity, wind direction, angle, wind speed and air volume on air conditioning efficiency are analyzed, and the relationship between indoor temperature and space-time change in summer and winter is simulated. At the same time, the shape optimization model of the air conditioner is determined, and the best shape and size of the air conditioner are designed to achieve the best temperature adjustment effect.

For the problem 2, in order to maximize the purification effect, according to the different shapes of the air purifier, the optimization model of the shape of the air purifier is established, the influence of the shape of the air purification on the purification effect is analyzed, and the best size and shape of the purifier is designed, and the best shape and size parameters are drawn.

For the 3 of the problem, in order to maximize the humidification effect, based on the different shapes of the air humidifier, the influence of the shape of the air humidifier on the humidification effect is analyzed, the air humidification shape optimization model is established, the best shape of the humidifier is designed, and the best shape and size parameters are drawn.

For the problem 4, in order to maximize the energy efficiency, human comfort and purification and humidification effect, based on the first three questions model, the optimization design model is established, the best appearance of the 3 equipment is designed, and the shape and size of the best appearance are drawn.

2 problem analysis

2.1 analysis of problem 1

This question requires the establishment of a model to simulate the change of indoor temperature with time and space in different seasons, establish an optimization model, and design the best shape and size of the air conditioner when the air conditioner reaches the maximum adjustment effect. Considering that the air flow generated by the air-conditioning gas in the flow process belongs to the compressible Newtonian fluid, the Navier-Stokes equation is established here to describe the air flow and then, with the uniformity of temperature, the efficiency of energy, the high control of noise as the target function, and the volume as the constraint to establish a single-target planning model. Then, the random search algorithm solves the above model, and determines the best shape and size of the air conditioner under the change of room temperature with time and space in summer and winter, as well as the best temperature regulation effect.

2.2 analysis of problem 2

This topic requires considering different shapes of air purifiers, maximizing the purification effect, designing the best shape and best parameters of the air humidifier, and visualizing them. Considering that the essence of an air purifier is to absorb tiny substances in the air, the diffusion of tiny substances can be described by the Adwex-diffusion equation. Then, the single-objective planning model is established with the optimal air purification effect as the target function and the volume limit as the

constraint. Then, the model is solved by using the gradient descent algorithm that makes the target function drop to the minimum value along the fastest path, and the best shape and size of the air purifier when the purification effect is optimal are calculated, and the visual description is made. This paper uses robustness analysis to further analyze the purification effect of air purifiers by changing the distribution of pollutants and changing the distribution from random distribution to normal distribution.

2.3 analysis of problem 3

This topic requires considering different shapes of air humidifiers, maximizing the humidification effect, designing the best shape and best parameters of the air humidifier, and visualizing them. The effect of air humidifier is mainly measured by humidity. The shape and size of air purification affect the release mode and distribution of water vapor. Then the water vapor diffusion equation is introduced here. Fick's law is introduced here to establish the water vapor diffusion equation to describe the water vapor concentration. Then, with the maximization of air humidification effect as the target function and the volume limit as the constraint, a single-goal planning model is established. And then,

The BFGS optimization algorithm of Newton method is used to solve the single-target planning model, and the best shape and size when the humidification effect is optimal are calculated. Finally, this paper uses sensitivity analysis to test the model and change the maximum power to calculate the degree of change in the purification effect.

2.4 analysis of problem 4

This question requires optimizing the appearance design of the 3 device on the basis of the 3 of the problem 1. 2., establishing an optimization model, designing the best appearance, and drawing the shape and size parameters of the best appearance. In this paper, the Pareto model is used to integrate the above-mentioned single-goal planning model, and the multi-objective planning model is established with the objective function of energy efficiency, human comfort, purification effect and humidification effect. Then, the machine learning-SLSQP algorithm is used to solve the above model, determine the best appearance shape and size parameters, and draw the best appearance shape.

3 Model assumptions

- Assuming that the air in the room is an incompressible ideal gas, the thermophysical properties of the air (such as specific heat capacity, thermal conductivity, etc.) are constant.
- Ignore the heat capacity of the wall and think that the wall temperature remains constant
- Assuming that the air flow blown by the air conditioner is evenly mixed in the room
- Assuming uniform temperature distribution in the initial time space

- Assuming that the air conditioner, air purifier and air humidifier will not affect each other when they work together
- Assuming that the temperature in the room is only affected by the air conditioning

4 Symbol Description

Symbol	Description	Unit
u	air circulation speed	Meters per second (m/s)
p	Air pressure	Pa (Pa)
μ	aerodynamic viscosity	Pa·s
T_{target}	Target temperature	Degrees Celsius (°C)
t	Time interval	Second (s)
C_w	Volume concentration of water vapor	Molar concentration (mol/m ³)
V_h	Volume of air humidifier	Cubic Meters (m ³)
a	Step size of learning rate	Unit value (1)
S	Surface area of humidifier	Square meters (square meters)
Q	Air volume	Cubic meters per second (m ³ /s)

5 Our Work

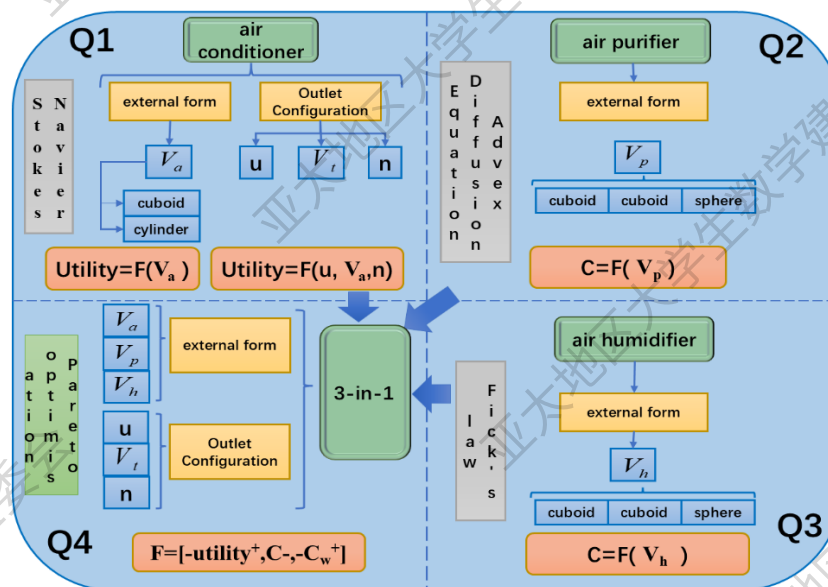


Figure 1 Our Work

The above figure shows the overall idea of solving the problem in the form of a flowchart, and determines the best solution to the problem by using the goal planning

model. The overall idea is from data processing to model construction to target planning model construction, and finally to determine the best solution and use the corresponding algorithm or directly use Matlab software to calculate, to get the answer. The specific process is shown below.

6 establishment and solution of the model

6.1 Model establishment and solution of problem 1

6.1.1 Establishment of air circulation equation

Air flow belongs to compressible Newtonian fluid, then the Navier-Stokes equation ^[1] is established here to describe the airflow, which is essentially momentum conservation and can be expressed:

$$\rho\left(\frac{\partial u}{\partial t} + u \cdot \nabla u\right) = -\nabla p + \nabla \cdot (\mu(\nabla u + (\nabla u)^T)) - \frac{2}{3}\mu(\nabla \cdot u)I + F \quad (1)$$

u Indicates the air circulation velocity, p the air pressure, ρ the air density, and μ the aerodynamic viscosity.

This equation is solved simultaneously with the continuity equation, I .e., mass conservation, expressed:

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho u) = 0 \quad (2)$$

To relate the Navier-Stokes equation to temperature, heat is introduced based on $Q = \nu R \Delta T$. For any object, the law of conservation of energy is satisfied. The length, width and height of the $5m \times 8m \times 3m$ room are taken as the base, a spatial rectangular coordinate system is established, and the microelements in the given fixed volume room are taken to meet the $Q_{\text{inf}} + Q_s = Q_r$. Q_{inf} is the inflow heat, Q_s is the heat source heat, and Q_r is the heat that increases the temperature. Taking into account the derived

$\vec{Q} = -k\Delta u$ from Fourier's law of heat conduction, that is, heat flows from high temperature to low temperature, the amount of heat flowing in a certain direction is proportional to the reduction rate of temperature in that direction, and it is substituted into heat conservation, that is, the heat transfer equation is derived:

$$\frac{\partial u}{\partial t} = \alpha^2 \Delta u + F(t, \vec{x}) \quad (3)$$

Where $\alpha = \sqrt{\frac{\kappa}{c\rho}}$, $F(t, \vec{x}) = \frac{g(t, \vec{x})}{c\rho}$, substituting (3), the heat transfer equation can

be simplified:

$$\frac{\partial u}{\partial t} = \frac{\kappa}{c\rho} \Delta u + \frac{g(t, \vec{x})}{c\rho} = \frac{1}{c\rho} (\kappa \Delta u + g(t, \vec{x})) \quad (4)$$

The κ in the formula is the thermal diffusion coefficient, the $g(x, y, z, t)$ represents the heat production rate of the internal heat source per unit volume, and the c is the specific heat capacity of the air.

Finally, the equation of (1)(2)(4) can describe the air flow and temperature change is obtained.

$$\rho c \frac{\partial T}{\partial t} + \rho \cdot c \cdot (\vec{u} \cdot \nabla T) = \kappa \nabla^2 T + Q_s \quad (5)$$

This equation establishes a connection between air density, specific heat capacity, temperature, flow velocity and heat source, which can be used as a theoretical basis for the design of air conditioning outlet configuration and air conditioning shape.

6.1.2 The establishment of high-performance air-conditioning optimization model.

Question given the relevant factors will be on the air conditioning working state to make a certain degree of adjustment, thus affecting the air conditioning refrigeration effect, this part will be 6.1.1 section of the establishment of the flow temperature differential equation [2] based on, respectively, on the air conditioning design to discuss.

The relevant factors given can be divided into five categories. The 1 category is the position of the air conditioner and the position of the air inlet. The different positions will determine the flow path of the air flow, thus affecting the uniformity of the temperature in a given room. The second category is the number of air inlets. The increase in the number of air inlets will improve the air circulation, but too many air inlets will also increase the difficulty of air conditioning design, so the advantages and disadvantages need to be considered in a balanced manner. The third category is the direction and angle of air inlets, it can determine the range that the air flow can cover, and a better angle can further improve the use effectiveness of the air conditioner. The fourth category is air volume, which determines the speed of air replacement, thus affecting the working efficiency of the air conditioner. The last category is wind speed, which should be moderate. If the wind speed is too high, it will lead to uncomfortable reaction of users, and noise will be generated when the wind speed is higher, which will affect the user's use feeling.

In order to simplify the model, the simplified design objectives are initially divided into two parts:

(1) air conditioning design

Shape and size of air conditioner

(2) Air outlet configuration design

Air conditioning position, air inlet position, quantity, wind direction, angle, wind speed and air volume

First of all, assume that the air outlet configuration is the same, only change the shape and size of the air conditioner, and based on market research, the shape of the air conditioner is mainly cylindrical and cuboid, then the shape will be discussed in these two shapes, so as to determine the most air conditioner shape and size, as shown in the

figure:



Fig. 2 Design Drawing of Left (Cylinder) and Right (Cuboid) Tuyere

After determining the optimal shape and size of the air conditioner, the sealing configuration is designed to determine the optimal air conditioner position, air inlet and outlet position, quantity, wind direction, angle, wind speed and air volume.

6.1.2.1 objective function

Optimizing the performance of air conditioning is carried out from the 3 aspects, the 1 is the uniformity of temperature, the 2 is the efficiency of energy, and the last is the high control of noise

(1) temperature uniformity

The temperature distribution of high-performance air conditioners in the temperature field should tend to be uniform, then the first goal is to construct: temperature uniformity, which can be expressed:

$$Temperature_uniformity = \frac{1}{V} \int |T(x, y, z) - T_{target}|^2 dV \quad (6)$$

The $T(x, y, z)$ represents the temperature of each coordinate in the temperature field ^[3] of the three-dimensional coordinate system of the room based on the air circulation model, and the T_{target} represents the target temperature set by the user for the air conditioner. The temperature uniformity of the air conditioner is expressed by the sum of the temperature differences in the temperature field of the whole room. The sum shows that the air conditioner as a whole minimizes the difference between the set temperature and the actual temperature at each position in the room, that is, the temperature uniformity is well performed.

The first 1, second, and third categories in the indicator will have a greater impact on the uniformity of temperature.

(2) Energy efficiency:

The second goal is the highest energy efficiency. The energy efficiency of air conditioning refrigeration and heating can be expressed in terms of cooling energy efficiency ratio (EER) and heating energy efficiency ratio (COP). Here it can be summarized as efficiency can be expressed:

$$\eta = \frac{Q_s / \Delta t}{P} \quad (7)$$

Q_s For the heat source, the cooling capacity is the cooling capacity, while the heating is the heating capacity, the Δt is the heat source action time, and the P is the air conditioning electric power. The fourth category of indicators will have a greater

impact on energy efficiency.

(3) High noise control

According to the data, the noise between wind speed and noise increases with the increase of wind speed and produces specific noise under specific wind speed. The shape of the air conditioner also affects its noise, considering that the design of the air conditioner in this paper is based on the existing air conditioner on the market to improve, then only consider the impact of wind speed at this time, in order to simplify the model, build the noise function and set up a coefficient based on this, expressed:

$$noise = \delta u + \varepsilon \quad (8)$$

Among them $\delta > 0$, this formula reflects the wind speed and noise size is positively correlated.

Taking the 3 characteristics as the target function, the multi-objective planning model is constructed, and in order to simplify the calculation difficulty of the model, the utility function of air conditioning is constructed here, and the multi-objective planning is transformed into a single-target planning model.

The utility function for air conditioning is expressed:

$$utility = \alpha_1 \eta - \alpha_2 Temperature_uniformity - \alpha_3 noise \quad (9)$$

Among them, α_1, α_2 , and α_3 are their weighting coefficients, and their Sanskrit 3 the priority of one indicator.

6.1.2.2 constraint condition

If the volume of the air conditioner is V_a , if it is a cuboid, its length, width and height are a , b and c respectively, it is $V_a = abc$. If it is a cylinder, the diameter is d , and it is $V_a = \frac{1}{4} \pi d^2 c$. If it is known that the maximum volume of the air conditioner is 0.1 cubic meters, it can be expressed as follows:

$$V_a \leq 0.1 \quad (10)$$

The power should be less than the rated power, and the wind speed should be less than the maximum wind speed, which is expressed:

$$P \leq P_e = 1800 \quad (11)$$

The wind speed shall be less than the maximum outlet speed:

$$u \leq 8 \quad (12)$$

Based on $c = \frac{Q_{inf}}{m \Delta T} = \frac{Q_{inf}}{\rho V_t \Delta T}$, where V_t is the airflow volume of unit tuyere,

assuming that each tuyere is equal in size, the number of seals is n , and the maximum inlet and outlet flow rate is 600 cubic meters, which can be expressed:

$$nV_t = \frac{Q_{inf}}{c\rho\Delta T} \leq 600 \quad (13)$$

6.1.2.3 establishment of optimization model

Based on the objective function and constraints, the goal is to maximize the efficiency of user use, while the decision variables are different in the two design steps and will be described separately. When designing the shape and size of the air conditioner, the V_a is used as the decision variable when changing the size and shape of the air conditioner, while the air inlet and outlet design will take the wind speed u , the air volume V_t of the unit air inlet and outlet, and the number of air outlets as the n , and the single-target planning model will be organized as follows:

$$\begin{aligned} & \min(-utility) \\ & \begin{cases} V_a \leq 0.1 \\ P \leq 1800 \\ u \leq 8 \\ nV_t \leq \frac{Q_{inf}}{c\rho\Delta T} \leq 0.6 \\ \eta = \frac{Q_s / \Delta t}{P} \end{cases} \\ & \text{s.t.} \begin{cases} utility = \alpha_1 \eta - \alpha_2 \text{Tempreture_uniformity} - \alpha_3 \text{noise} \\ \text{noise} = \delta u + \varepsilon \\ \text{Tempreture_uniformity} = \frac{1}{V} \int |T(x, y, z) - T_{target}|^2 dV \\ \rho c \frac{\partial T}{\partial t} + \rho \cdot c \cdot (\vec{u} \cdot \nabla T) = \kappa \nabla^2 T + Q_s \\ Q_{inf} + Q_s = Q_T \end{cases} \end{aligned} \quad (14)$$

6.1.3 The solution of the optimization model of high-performance air conditioning.

The above target planning model is solved by random search algorithm, and the basic principle of the algorithm is to randomly sample the 1 group parameters in a given parameter space and calculate the target function value of the parameter combination. Through continuous random sampling and evaluation, the best parameter combination that makes the target function reach the minimum or maximum value is finally found.

Because the model contains differential equations, this paper uses the finite difference method^[4] to optimize the solution of the model on the basis of random search algorithm. The finite difference method is the numerical solution of the 1 differential equation, which is one of the most basic and most extensive numerical methods for time dispersion and space dispersion, and is widely used in the fields of computational fluid mechanics, computational heat transfer, computational electromagnetics and so on.

Taking a one-dimensional problem as an example, suppose there is a function $f(x)$ about x , using Taylor expansion:

$$f(x + \Delta x) = f(x) + \frac{\Delta x}{1!} \frac{\partial f}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 f}{\partial x^2} + O(\Delta x^3) \quad (15)$$

The 1-order derivative that ignores the remainder can be expressed:

$$\frac{\partial f}{\partial x} \approx \frac{f(x + \Delta x) - f(x)}{\Delta x} \quad (16)$$

This format is called Euler forward difference, and because of the 1 order function of the ignored term x , it has 1 order accuracy and is expected to be truncated error.

Similarly, Euler backward difference is;

$$\frac{\partial f}{\partial x} \approx \frac{f(x) - f(x + \Delta x)}{\Delta x} \quad (17)$$

Euler's backward difference also has the 1-order accuracy, in order to build the second-order accuracy, first, the above two sets of equations are expanded with Taylor:

$$\begin{cases} f(x + \Delta x) = f(x) + \frac{\Delta x}{1!} \frac{\partial f}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 f}{\partial x^2} + \frac{\Delta x^3}{3!} \frac{\partial^3 f}{\partial x^3} + O(\Delta x^4) \\ f(x - \Delta x) = f(x) - \frac{\Delta x}{1!} \frac{\partial f}{\partial x} + \frac{\Delta x^2}{2!} \frac{\partial^2 f}{\partial x^2} - \frac{\Delta x^3}{3!} \frac{\partial^3 f}{\partial x^3} + O(\Delta x^4) \end{cases} \quad (18)$$

Subtract the two, ignoring the remainder, and get:

$$\frac{\partial f}{\partial x} \approx \frac{f(x + \Delta x) - f(x - \Delta x)}{2\Delta x} \quad (19)$$

The above formula is the central differential format of second-order accuracy.

6.1.4 Seasonal simulation

In order to further simulate the air-conditioning gas fluid process in summer and winter, this paper uses Lagrangian CFD ^[5] to describe the fluid motion state. The Lagrangian method focuses on the study of the motion of each fluid particle, describing the process of motion of the fluid particle from beginning to end, and the law of change of physical quantities with time t .

$$\nabla = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right) = \frac{\partial}{\partial x} i + \frac{\partial}{\partial y} j + \frac{\partial}{\partial z} k \quad (20)$$

Velocity gradient:

$$\nabla V = \left(\frac{\partial u_x}{\partial x}, \frac{\partial u_y}{\partial y}, \frac{\partial u_z}{\partial z} \right) \quad (21)$$

divergence:

$$\nabla V = \left(\frac{\partial}{\partial x}, \frac{\partial}{\partial y}, \frac{\partial}{\partial z} \right) \cdot (u_x, u_y, u_z) = \frac{\partial u_x}{\partial x} + \frac{\partial u_y}{\partial y} + \frac{\partial u_z}{\partial z} \quad (22)$$

Rotary:

$$\nabla V = \begin{vmatrix} i & j & k \\ \frac{\partial}{\partial x} & \frac{\partial}{\partial y} & \frac{\partial}{\partial z} \\ u_x & u_y & u_z \end{vmatrix} = ((\frac{\partial u_z}{\partial y} - \frac{\partial u_y}{\partial z})i, (\frac{\partial u_x}{\partial z} - \frac{\partial u_z}{\partial x})j, (\frac{\partial u_y}{\partial x} - \frac{\partial u_x}{\partial y})k) \quad (23)$$

Acceleration:

The local acceleration caused by the unsteady flow and the displacement acceleration caused by the non-uniform flow are composed of two parts.

$$a = \frac{\partial u_y}{\partial t} + (\nabla V)V \quad (24)$$

The optimal volume of the air conditioner and the temperature setting conditions of the air conditioner in summer and winter are determined by the above random search algorithm. The following table shows the temperature setting in summer and winter, as shown in Figure 3 and Figure 4.

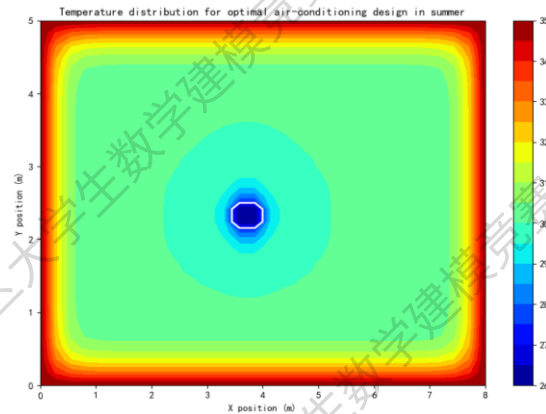


Fig. 3 Air conditioning temperature setting in summer

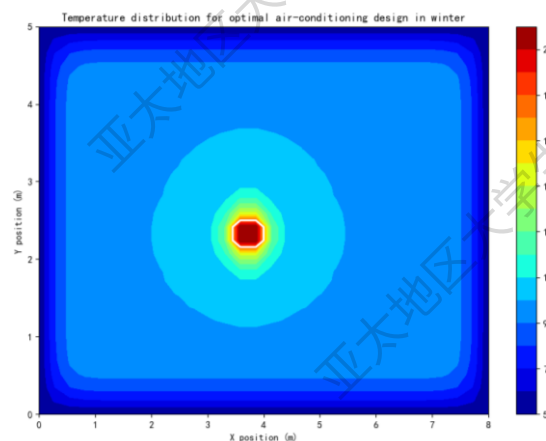


Fig. 4 Air conditioning temperature setting in winter

This article shows the optimal volume setting of the air conditioner and the temperature utility here:

Table 1 1 Best state of air conditioning

Status	Winter	Summer
--------	--------	--------

Long	0.5	0.5
Wide	0.5	0.5
High	0.5	0.5
utility	7.84	7.93

The air conditioner shown above is in the best state, and the best shape of the air conditioner is cuboid. At this time, the best volume of the air conditioner in winter and summer is $3 \times 0.75\text{m}$. Among them, the summer air conditioning design is more effective and the temperature adjustment effect is better.

Taking into account the air conditioning is affected by the location, air inlet location, quantity, wind direction, angle, wind speed and air volume and other factors, the indoor temperature changes with time and space under different conditions in summer and winter are simulated. This paper analyzes the change of indoor temperature from 3 angles of air outlet, wind speed, number of air conditioners and placement position. The specific change is shown in the following figure:

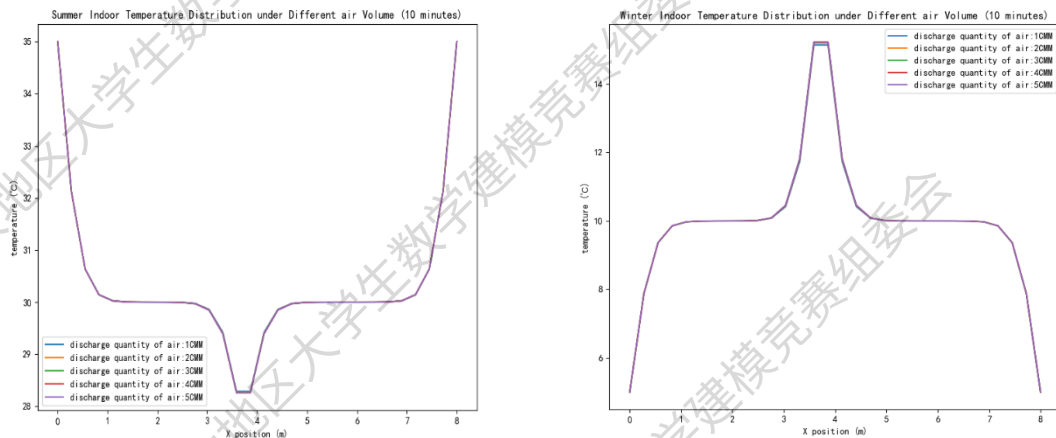
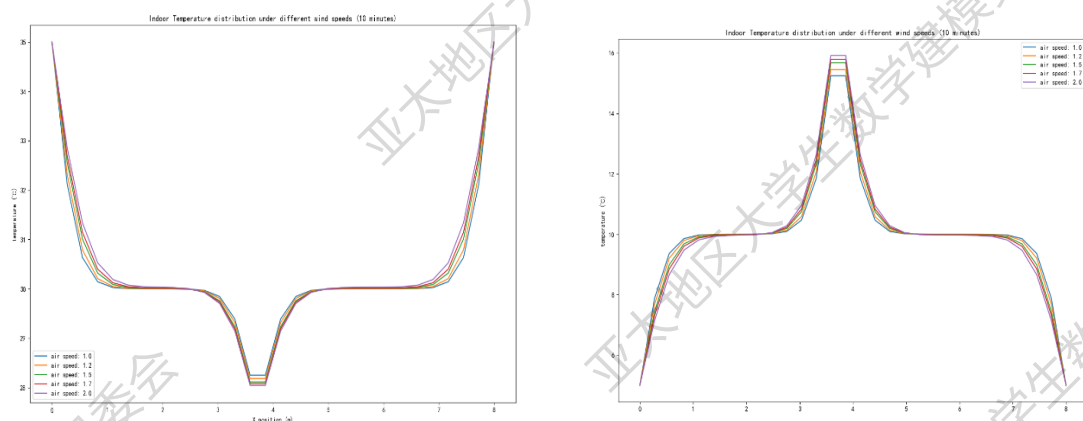


Fig. 5 Influence of air output on temperature in summer (left) and winter (right)

In this paper, summer and winter as the sub-point, the air volume in accordance with $1\text{--}5\text{m}^3/\text{s}$ respectively to simulate the temperature changes, the results show that



in the two seasons of summer and winter, the air volume and temperature changes, the basic coincidence, indicating that the air volume on the air conditioning temperature effect is small.

Figure 6 Influence of wind speed on temperature in summer (left) and winter (right)

In this paper, summer and winter are used as sub-points, the air output is 1 m/s as the initial speed, 0.2 m/s as the step size, and the temperature change is simulated

respectively. The results show that in the two seasons of summer and winter, the change of wind speed and temperature can be clearly seen that the change trend is different, indicating that the air output has obvious influence on the temperature effect of air conditioning.

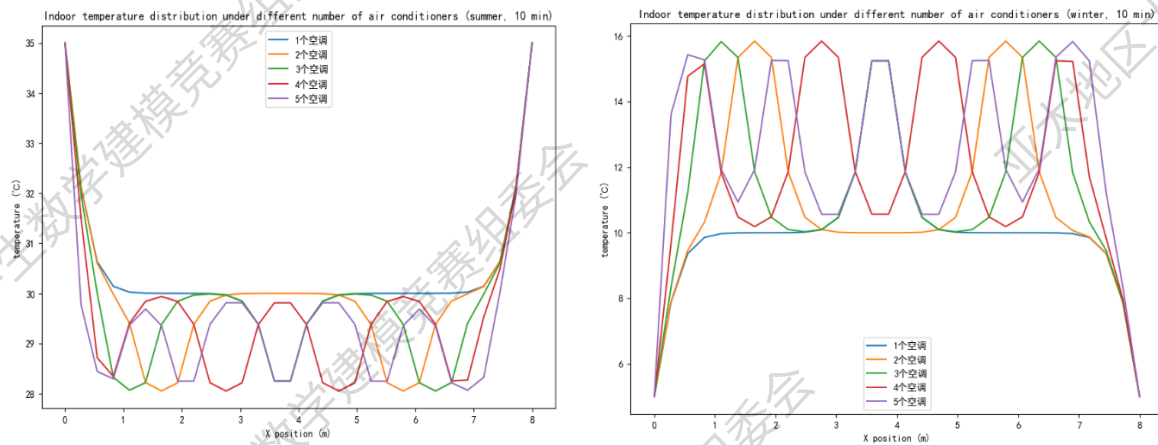


Fig. 7 Change of air conditioning quantity on temperature in summer (left) and winter (right)

In this paper, the summer and winter points, respectively, the number of air conditioning 1-5 as the research object, the temperature change simulation, the results show that in the summer and winter two seasons, the number of air conditioning and temperature changes, it can be clearly seen that there is a big difference in the trend, indicating that the number of air conditioning on the temperature effect of air conditioning.

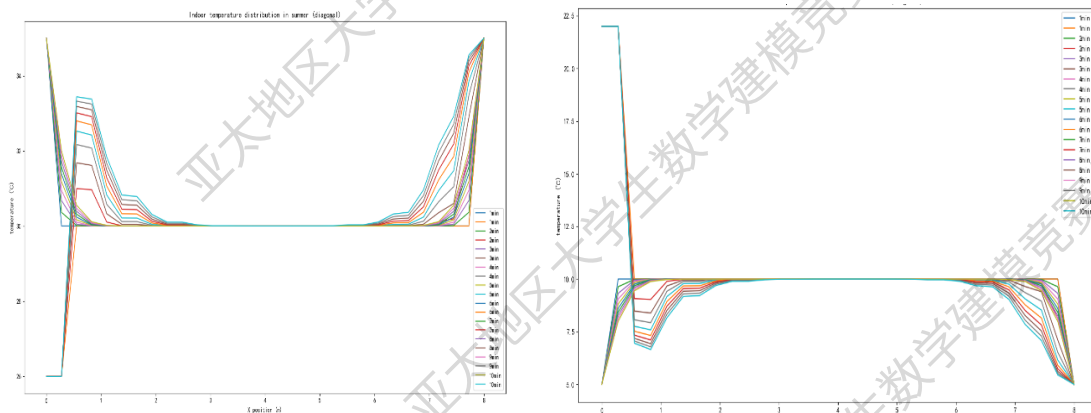


Figure 8 Summer (left) and winter (right) air conditioning position on the diagonal changes in temperature

In this paper, summer and winter are taken as sub-points, and the time span of 1-10 minutes of the number of air conditioners is taken as the research object to simulate the temperature change. The results show that in the two seasons of summer and winter, the position and temperature change of each air conditioner on the diagonal of the room can be clearly seen that there are obvious differences in the change trend, which shows that the temperature effect produced by the air conditioner is greater when the position on the diagonal.

In this paper, summer and winter as the points, respectively, 1-10 time span as the research object, the temperature change simulation, the results show that in the summer and winter two seasons, each air conditioning position in the middle and the

temperature change, it can be clearly seen that there are obvious differences in the trend of change, indicating that the number of air conditioning on the temperature effect of air conditioning is more obvious.

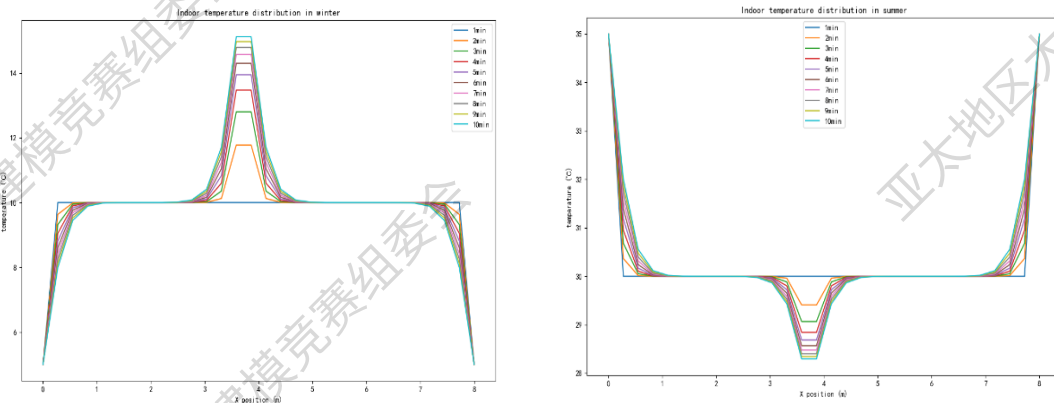


Fig. 9 Change of temperature affected by air conditioning position in the middle of summer (left) and winter (right)

To sum up, the relevant factors that have a greater impact on the change of the air conditioner temperature are the wind speed, quantity and spatial location of the air conditioner.

6.1.5 Summary

This paper considers the air flow generated during the operation of the air conditioner and establishes the air circulation equation. On this basis, the optimization model of high performance air conditioning is established. On the basis of the random search algorithm, the model is optimized by using the finite difference method, and the best shape of the air conditioner is determined to be a cuboid, and the best size is 0.5m x 0.5m x 0.5m. Then, the Lagrange algorithm is used to solve the best temperature adjustment effect of air conditioning in summer and winter, and the air conditioning adjustment effect in summer is better than that in winter.

6.2 Model establishment and solution of problem 2

6.2.1 Establishment of air purification model

The essence of an air purifier ^[6] is to absorb tiny substances in the air, and the diffusion of tiny substances can be described by the transmission diffusion equation. The Adwex-diffusion equation considers the transmission of molecular diffusion caused by fluid motion and concentration gradient, and can more comprehensively describe the transmission and diffusion process of substances in multi-dimensional space, which can be expressed as follows:

$$\frac{\partial C}{\partial t} + u_x \frac{\partial C}{\partial x} + u_y \frac{\partial C}{\partial y} + u_z \frac{\partial C}{\partial z} = D_x \frac{\partial^2 C}{\partial x^2} + D_y \frac{\partial^2 C}{\partial y^2} + D_z \frac{\partial^2 C}{\partial z^2} \quad (25)$$

Among them, D_x , D_y and D_z are the diffusion coefficients in the 3 coordinate system, C is the substance concentration, and u_x , u_y and u_z are the air velocity in the

3 coordinate system.

For the solution of the diffusion coefficient, this question will use the mean square displacement method, which can be expressed:

$$\begin{cases} D = \frac{\langle r^2(t) \rangle}{6t} \\ r = \sqrt{x^2 + y^2 + z^2} \end{cases} \quad (26)$$

Where $\langle r^2(t) \rangle$ represents the mean square displacement and t is the time interval, the diffusion coefficient is estimated by fitting the slope of the leading part.

This partial differential equation establishes a three-dimensional coordinate system in a room $5m \times 8m \times 3m$ by reference to problem 1. Different positions correspond to different substance concentration $C(x, y, z)$, and all coordinates constitute a substance concentration field. In order to simplify the model, combined with energy conservation, it is transformed into a differential equation, expressed:

$$dC = CV_p dt - SQdt \quad (27)$$

Among them, S is the surface area, Q is the air volume, and V_p is the volume of the air purifier. Similar to the air conditioner, the existing air purifiers in the market are mostly cuboid, cylindrical, and spherical, which will be discussed separately here. For cuboids, $V_p = abc$, $S = 2(ab + ac + bc)$, where a , b , and c are the length, width and height of the air purifier respectively; for cylinders, $V_p = \frac{1}{4}\pi d^2 c$, $S = \frac{1}{2}\pi d^2 + \pi dc$, where d is the diameter of the cylinder, and c is the height of the cylinder; for spheres, $V_p = \frac{4}{3}\pi r^3$, $S = 4\pi r^2$. $SQdt$ is the minimum value of pollutant concentration per unit time.

simultaneous (15), (17), available

$$(CV_p - SQ) + u_x \frac{\partial C}{\partial x} + u_y \frac{\partial C}{\partial y} + u_z \frac{\partial C}{\partial z} = D_x \frac{\partial^2 C}{\partial x^2} + D_y \frac{\partial^2 C}{\partial y^2} + D_z \frac{\partial^2 C}{\partial z^2} \quad (28)$$

At this time, the partial differential equation (18) links the time dimension factor with the spatial dimension, which can describe the material concentration state of each volume element in a given space, and provide a theoretical basis for the subsequent model establishment.

6.2.2 Establishment of Optimization Model for High Performance Air Purification

6.2.2.1 objective function

This topic is mainly based on the optimization of air purifiers, and the shape and size of air purifiers are designed to make them high-performance air purifiers.

According to relevant data, the high-performance air purifier is mainly reflected

[7]by the concentration of substances in the space, then the model here is based on the diffusion equation to construct the relationship between different coordinates and concentrations in the space field, and ultimately to ensure that the total concentration of substances in a given size is the smallest in the $5m \times 8m \times 3m$ space, at this time the air purifier effect is optimal. Then sum the concentrations of substances in space, expressed:

$$C = \int_V dC \quad (29)$$

The microelement form of the substance concentration is shown in (17).

6.2.2.2 constraint condition

The volume of the air purifier cannot exceed the size of a given space, which can be expressed:

$$V_p \leq 120 \quad (30)$$

Among them V_p is the volume of the air purifier, and the mention of different shapes needs to be discussed separately.

6.2.2.3 establishment of optimization model

In summary, with the minimum concentration of space material as the target function, the volume of air purifier as the decision variable, the establishment of a single-goal planning model, can be expressed:

$$\begin{aligned} \min C & \\ \left\{ \begin{array}{l} C = \int_V dC \\ (CV_p - SQ) + u_x \frac{\partial C}{\partial x} + u_y \frac{\partial C}{\partial y} + u_z \frac{\partial C}{\partial z} = D_x \frac{\partial^2 C}{\partial x^2} + D_y \frac{\partial^2 C}{\partial y^2} + D_z \frac{\partial^2 C}{\partial z^2} \\ D = \frac{\langle r^2(t) \rangle}{6t} \end{array} \right. & \\ \text{s.t.} \left\{ \begin{array}{l} V_{pi} = abc, S_i = 2(ab + ac + bc), i \in \text{cuboid} \\ V_{pi} = \frac{1}{4} \pi d^2 c, S = \frac{1}{2} \pi d^2 + \pi dc, i \in \text{cylinder} \\ V_{pi} = \frac{4}{3} \pi r^3, S = 4\pi r^2, i \in \text{sphere} \\ V_p \leq 120 \end{array} \right. & \end{aligned} \quad (31)$$

6.2.3 Solution of high performance air purification optimization model

To solve the above model, this question is calculated using the gradient descent algorithm^[8]. The gradient descent algorithm is aimed at the lowest optimization problem, the goal is to make the objective function along the fastest path down to the minimum.

First, the cost function of the mean square error is added to the formula of the least squares, and it is linked to eliminate the effect of the number of samples on the parameter solution, then the partial derivative of the function is:

$$\frac{\delta(J(\theta))}{\delta(\theta_j)} = -\frac{1}{n} \sum_{i=1}^n [(h_{\theta}(x_i) - y_i)x_{ij}] \quad (32)$$

Next, using the gradient descent rule described earlier, we obtain:

$$\theta_j = \theta_{(j-1)} - a \frac{1}{n} \sum_{i=1}^n [h_{\theta}(x_i) - y_i)x_{ij}] \quad (33)$$

where n denotes the number of samples, a denotes the step size of the learning rate, and $h(x)$ denotes the cost function.

By using the gradient descent algorithm to solve the high-performance air purification optimization model, it is determined that the best shape of the air purifier is cuboid, the best parameters are length = 0.82 meters, width = 0.92 meters, height = 1.18 meters, at this time, the purification effect of the air purifier is visualized, as shown in the following figure:

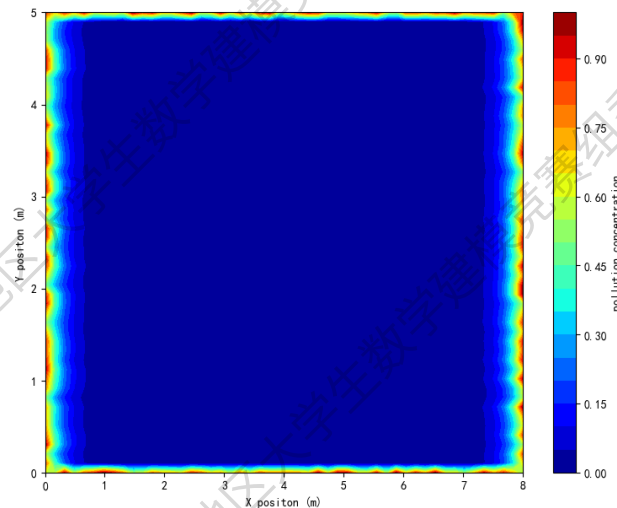


Fig. 10 Visualization of purification effect

As can be seen from the above figure, the air purifier has a high degree of purification, and most of the room space is in a blue state, indicating that the cleanliness is high. Therefore, the above figure is the state analysis diagram with the highest degree of purification in this topic.

6.2.4 inspection and analysis

This paper uses robustness analysis to test the model. The following figure shows the distribution state of pollutant concentration. It can be seen from the figure that the pollutant concentration shows a random distribution state at this time. In this regard, this paper further analyzes the purification effect of air purifiers by changing the distribution of pollutants and changing the distribution from random distribution to normal distribution.

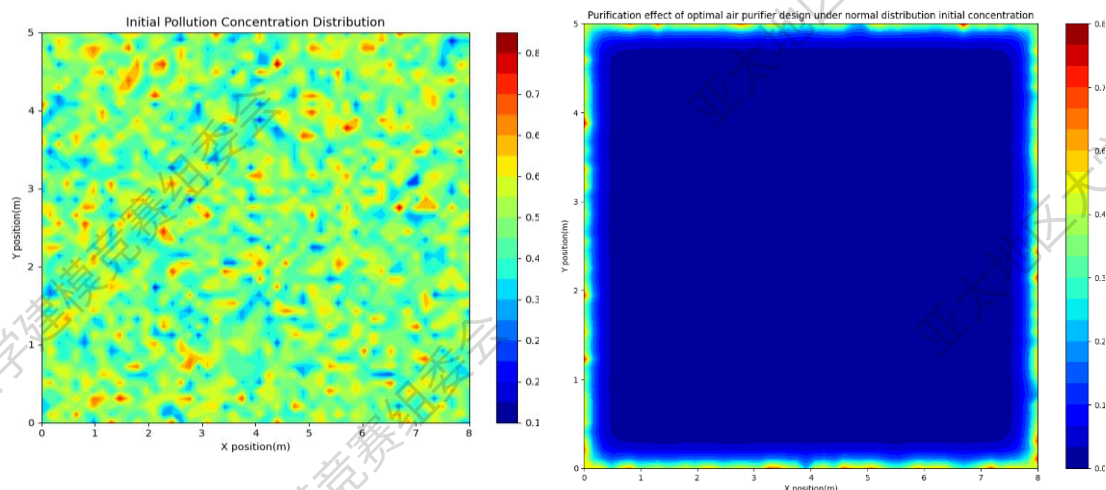


Figure 11 Distribution of initial pollutant concentration(left) Purification effect of normal distribution initial concentration of the best air purifier design(right)

The above figure shows the purification effect of the air purifier after changing the distribution state. It can be seen from the figure that the air purification effect is the same as the initial state at this time, and the purification effect is better.

6.2.5 Summary

Considering that the essence of air purifiers is to absorb tiny substances in the air, this question selects the Adwex-diffusion equation to comprehensively describe the transmission and diffusion process of substances in multidimensional space. On this basis, this topic takes the best purification effect as the target function and establishes the target planning model. Then, using the gradient descent algorithm, the above-mentioned objective function is solved, and the best shape of the air purifier in this question is the cuboid, the best size is the best parameter for the length of 0.82 meters, the width of 0.92 meters, the height of 1.18 meters, at this time the air purification effect is the best. Finally, the model is tested by robustness analysis, and the concentration distribution of pollutants is changed, and the purification effect is still very good, which shows that the accuracy of the model is high.

6.3 Model establishment and solution of problem 3

6.3.1 Establishment of Water Vapor Diffusion Model

The effect of air humidifier^[9] is mainly measured by humidity. The shape and size of air purification affect the release mode and distribution of water vapor. The water vapor diffusion equation is introduced here. Fick's law is introduced here to establish the water vapor diffusion equation to describe the water vapor concentration.

First of all, based on Fick's first law to establish the water vapor concentration vector field, the first law draws the conclusion that the diffusion material flow per unit time through the unit cross-sectional area perpendicular to the diffusion direction (called the diffusion flux j) is proportional to the concentration gradient at the cross-section, which can be expressed:

$$j = -D \frac{dC_w}{dx} \quad (34)$$

The D is the diffusion coefficient, the C_w is the volume concentration of water vapor, the $\frac{dC_w}{dx}$ represents the concentration gradient, and the negative sign indicates that the diffusion direction is opposite to the concentration gradient, that is, the diffusion element flows from high concentration to low concentration. It is then extended to a given three-dimensional space, which forms a humidity vector field, which can be expressed:

$$j = -D \nabla C_w \quad (35)$$

where j is the diffusion flux and ∇ is the gradient operator.

Considering non-steady diffusion, Fick's second law is introduced. At distance x , the rate of change of concentration with time is equal to the negative value of the rate of change of diffusion flux with distance, which can be expressed:

$$\frac{\partial C_w}{\partial t} = \frac{\partial}{\partial x} \left(D \frac{\partial C_w}{\partial x} \right) \quad (36)$$

where the diffusion coefficient D is constant and can be expressed in three-dimensional space after simplification as follows:

$$\frac{\partial C_w}{\partial t} = D \nabla^2 C_w \quad (37)$$

Among them ∇^2 , the Laplace operator, the estimation of the diffusion coefficient is consistent with the solution of the problem 2, using the mean square displacement method to estimate, (17) and (25) joint, then the water vapor concentration equation can be obtained, expressed:

$$SQ = D_x \frac{\partial^2 C_w}{\partial x^2} + D_y \frac{\partial^2 C_w}{\partial y^2} + D_z \frac{\partial^2 C_w}{\partial z^2} \quad (38)$$

Where S represents the surface area of the humidifier, Q represents the discharge amount of the humidifier, and SQ is the humidification humidity reduction value. Based on the market survey, still in the market mainstream air humidifier as a reference, respectively, the cuboid, cylinder, sphere shape to discuss. For the surface area of a cuboid, $S = 2(ab + ac + bc)$, where a , b , and c each represent the length, width and height of the cuboid; for a cylinder, $S = \frac{1}{2} \pi d^2 + \pi dc$, where d is the diameter of the cylinder and c is the height of the cylinder; for a sphere, $S = 4\pi r^2$, where r is the radius of the sphere.

6.3.2 Establishment of Optimization Model for High Performance Humidifier

6.3.2.1 objective function

Here, only from the size and size of the air moisture device to adjust, the highest concentration of water vapor in the air, that is, the performance of the air purifier is better, then the room who vapor concentration can be expressed;

$$C_w = \int_V dC_w \quad (39)$$

6.3.2.2 constraint condition

The constraint here is only that its volume cannot exceed the volume of the room, which can be expressed:

$$V_h \leq 120 \quad (40)$$

Among them V_h , the volume of the air humidifier, different collision air humidifier needs to be discussed separately.

6.3.2.3 establishment of optimization model

The high-performance air humidifier takes the maximum water vapor concentration in the field as the target function, takes the humidifier surface area S as the decision variable, and establishes a single-target planning model, which can be expressed:

$$\begin{aligned} & \min(-C_w) \\ & \begin{cases} C_w = \int_V dC_w \\ SQ = D_x \frac{\partial^2 C_w}{\partial x^2} + D_y \frac{\partial^2 C_w}{\partial y^2} + D_z \frac{\partial^2 C_w}{\partial z^2} \\ D = \frac{\langle r^2(t) \rangle}{6t} \end{cases} \\ & s.t. \begin{cases} V_{hi} = abc, S_i = 2(ab + ac + bc), i \in \text{cuboid} \\ V_{hi} = \frac{1}{4} \pi d^2 c, S = \frac{1}{2} \pi d^2 + \pi dc, i \in \text{cylinder} \\ V_{hi} = \frac{4}{3} \pi r^3, S = 4\pi r^2, i \in \text{sphere} \\ V_h \leq 120 \end{cases} \end{aligned} \quad (41)$$

6.3.3 Solution of Optimization Model of High Performance Humidifier

6.3.3.1 BFGS Optimization Algorithm of Quasi-Newton Method

For the above-mentioned single-target optimization model, the BFGS

optimization algorithm of Newton method is used to solve the model. The working principle of the BFGS optimization algorithm of the Newton method is to calculate the optimal result by iterating the algorithm, but the approximate matrix is always positive, so the algorithm is always searching in the direction of optimization, with global convergence and super-linear convergence speed. The BFGS algorithm uses more quasi-Newton methods, so it is called BFGS correction.

Correction on the basis of the quasi-Newton method:

Let $E_k = au_k v_k^T + \beta u_k v_k^T$, where u_k, v_k are all vectors of $n * 1$.

It can be resolved:

$$B_{k+1} s_k = y_k \quad (42)$$

Substituting the $E_k = au_k v_k^T + \beta u_k v_k^T$ into the above formula:

$$B_k + au_k v_k^T + \beta u_k v_k^T s_k = y_k \quad (43)$$

The final BFGS correction formula is:

$$B_{k+1} = B_k - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} + \frac{y_k y_k^T}{y_k^T s_k} \quad (44)$$

Through the of BFGS optimization algorithm ^[10] using quasi-Newton method, the single-objective planning model established for the above-mentioned high-performance air humidifier is solved by taking the maximum water vapor concentration in the field strength as the objective function and the humidifier surface area S as the decision variable, and the optimal shape of the air humidifier is rectangular, the size parameter length is 1.26m, the width is 0.2m, and the height is 0.5m. At this time, the humidification time can reach 20 hours and the humidification effect is maximized. The humidification status of the air humidifier is shown in the figure below. It can be seen from the figure that the closer the distance from the air purifier in the room, the better the air purification effect. Overall, the purification effect of this topic is better overall.

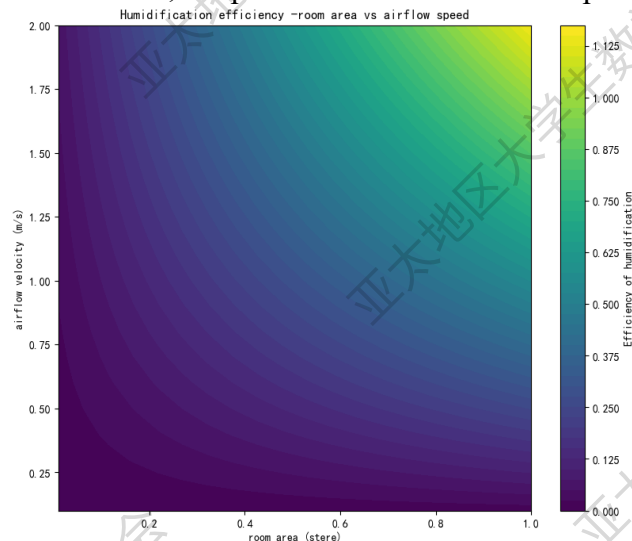


Figure 12 Air humidifier effect drawing

6.3.4 inspection and analysis

In order to further verify the accuracy of the model, this paper uses sensitivity analysis to test this question. In this paper, the parameters such as the shape, size, airflow speed and humidification time of the air humidifier are fixed, and the corresponding maximum humidification efficiency is calculated by changing the value of the maximum power.

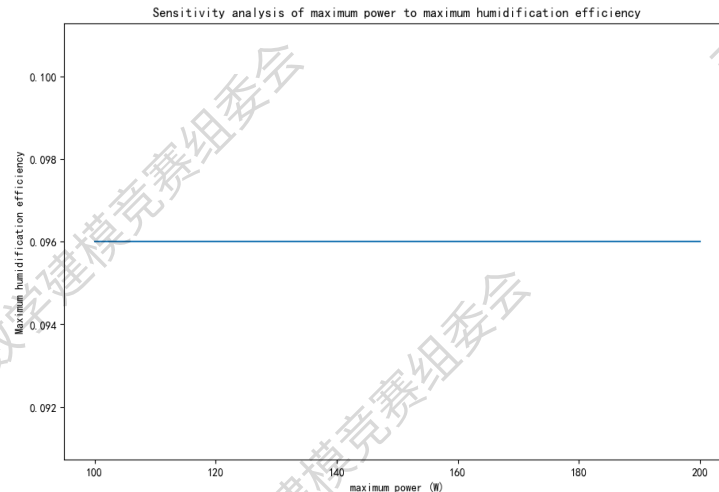


Figure 13 Sensitivity

With the maximum power of 100W as the initial value, with 1 as the step, to the end of 300W, calculate the best humidification efficiency, as shown in the figure, with the increase of power, the humidification rate is almost no change, indicating that the model is more stable.

6.3.5 Summary

Since the effect of air humidifier is mainly measured by humidity, Fick's law is introduced here to establish the water vapor diffusion equation to describe water vapor. Then, the single-objective planning model is established with the maximization of steam concentration as the target function and the volume condition as the constraint. In this paper, the BFGS optimization algorithm of Newton method is used to solve the above single-target planning model, and the best shape of the humidifier is rectangular, the size parameter length is 1.26m, the width is 0.2m, and the height is 0.5m. Finally, the model is tested by sensitivity analysis, and the results show that with the increase of power, the humidification rate has no change, indicating that the model has a high degree of stability.

6.4 Model establishment and solution of problem 4

6.4.1 The establishment of the Pareto optimization model.

It is necessary to establish a high-performance three-in-one equipment based on the problem 1. 2. three, and a new equipment with 3 functions of air conditioner, air purifier, air humidifier, and meet the optimal solution of the 3 optimization model at the same time. Considering that non-dominant individuals can be excluded in the Pareto optimization process to obtain a more convergent solution, that is, the optimal solution, the Pareto optimization model is introduced here.

Firstly, the above model is 1 to the model 3 objective function for classification, which is divided into positive function indicators and negative function indicators. For air conditioners, the higher the utility value, the better the performance, which is a positive function index, then it is $utility^+$. For air purifiers, the lower the sum of substance content and concentration in the room, the better the performance, that is, the negative function index, which is C^- . For air humidifier, the greater the sum of water vapor content and concentration in the room, the better its performance, that is, C_w^+ .

Here to maximize the performance of the three-and-one sense regulator as the goal, to air conditioning volume V_a , air purifier volume V_p , air humidifier volume V_h , wind speed u , unit air inlet air outlet air volume V_t , the number of air outlets for the n as the decision variable, multi-objective optimization model can be written:

$$\min F = [-utility^+, C^-, -C_w^+]$$

$$\begin{cases} V_a \leq 0.1 \\ P \leq 1800 \\ u \leq 8 \\ nV_t \leq \frac{Q_{inf}}{c\rho\Delta T} \leq 0.6 \\ V_p \leq 120 \\ V_h \leq 120 \\ C^- = \int_V dC^- \\ C_w^+ = \int_V dC_w^+ \\ SQ = D_x \frac{\partial^2 C_w^+}{\partial x^2} + D_y \frac{\partial^2 C_w^+}{\partial y^2} + D_z \frac{\partial^2 C_w^+}{\partial z^2} \\ (C^- V_p - SQ) + u_x \frac{\partial C^-}{\partial x} + u_y \frac{\partial C^-}{\partial y} + u_z \frac{\partial C^-}{\partial z} = D_x \frac{\partial^2 C^-}{\partial x^2} + D_y \frac{\partial^2 C^-}{\partial y^2} + D_z \frac{\partial^2 C^-}{\partial z^2} \quad (45) \\ s.t. \begin{cases} D = \frac{\langle r^2(t) \rangle}{6t} \\ V_{pi, hi} = abc, S_i = 2(ab + ac + bc), i \in \text{cuboid} \\ V_{pi, hi} = \frac{1}{4}\pi d^2 c, S = \frac{1}{2}\pi d^2 + \pi dc, i \in \text{cylinder} \\ V_{pi, hi} = \frac{4}{3}\pi r^3, S = 4\pi r^2, i \in \text{sphere} \\ \eta = \frac{Q_s / \Delta t}{P} \\ utility = \alpha_1 \eta - \alpha_2 \text{Tempreture_uniformity} - \alpha_3 \text{noise} \\ \text{noise} = \delta u + \varepsilon \\ \text{Tempreture_uniformity} = \frac{1}{V} \int |T(x, y, z) - T_{target}|^2 dV \\ \rho c \frac{\partial T}{\partial t} + \rho \cdot \mathbf{c} \cdot (\bar{\mathbf{u}} \cdot \nabla T) = \kappa \nabla^2 T + Q_s \\ Q_{inf} + Q_s = Q_r \end{cases} \end{cases}$$

6.4.2 solution of the model

In this paper, the SLSQP algorithm^[11] is used to solve the above multi-objective planning model. Sequential least squares is the 1 numerical method to solve unconstrained nonlinear optimization problems^[12]. Based on the idea of serialization, it decomposes the problem into a series of sub-problems to solve. In each subproblem, it is solved by minimizing a quadratic function.

By using machine learning-SLSQP algorithm to solve the above multi-objective planning model, the best three-in-one equipment is determined. Its best size is air conditioner: width = 1.24m, depth = 1.24m, and height range of air conditioner: 1.5~2.15m; Air purifier: width = 1.24m, depth = 1.24m, height range of humidifier: 0.85~1.5m; Humidifier: width = 1.24m, depth = 1.24m, height range of air purifier: 0~0.85m; According to this, the best appearance shape of the equipment can be drawn in this paper, as shown in the following figure:

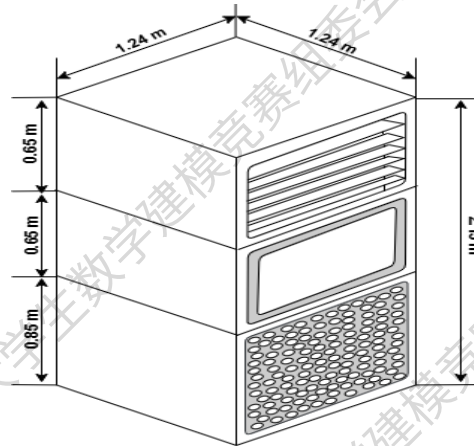


Fig. 14 Best Design

6.4.3 Summary

Based on the single-goal planning model established by the problem 1. 2. problem 3, this question adds energy efficiency, human comfort, purification effect and humidification effect maximization as the target function, and establishes a multi-objective planning model. Then, the machine learning-SLSQP algorithm is used to solve the multi-objective planning model, and the optimal parameters for the integration of 3 and equipment are obtained as follows: width = 1.24m, depth = 1.24m, and height range of air conditioner: 1.5~2.15m; Air purifier: width = 1.24m, depth = 1.24m, height range of humidifier: 0.85~1.5m; Humidifier: width = 1.24m, depth = 1.24m, the height range of air purifier: 0~0.85m, and draw the best appearance design.

7 evaluation and generalization of the model

7.1 evaluation of the model

7.1.1 Advantages of the model

- In designing and optimizing air conditioners, air purifiers, and humidifiers, this paper adopts a multi-objective planning model encompassing temperature uniformity,

energy efficiency, and noise control. This multi-objective design meets diverse usage scenarios and enhances overall equipment performance.

- Mathematical models such as Navier-Stokes and Advection-Diffusion equations are established to detail airflow and material diffusion processes, based on physical laws, ensuring high accuracy and reliability in simulating device operations in real-world environments.
- Various numerical methods, including random search, finite difference, gradient descent, and BFGS quasi-Newton optimization, are utilized to solve complex problems, leveraging their strong computational power and convergence for rapid optimal solution discovery.
- Robustness and sensitivity analyses validate model stability and accuracy under different conditions, exemplified by testing air purifier efficacy through varying pollutant concentrations and humidifier performance stability through altering maximum power values.

7.1.2 Disadvantages of the model

- The model assumes room air as incompressible ideal gas, neglecting practical factors like wall thermal capacity, which simplifies calculations but may lead to discrepancies with reality. It focuses on parameters like temperature, humidity, and air quality, with limited consideration for other environmental factors (e.g., lighting, wind speed variations) that could significantly impact device performance in real-world settings. To enhance model accuracy and practicability, more practical factors such as wall thermal capacity and indoor lighting changes can be incorporated.
- Device size and shape designs presented in the article are based primarily on theoretical calculations and simulations, potentially limited by practical constraints like installation space and power supply. Furthermore, device durability and long-term operational stability require further validation. Advanced optimization algorithms (e.g., Particle Swarm Optimization) can be employed to address conflicts in multi-objective optimization, seeking better trade-off solutions. Additionally, more constraints can be introduced to ensure design feasibility in real-world environments.

7.2 generalization of the model

- The model in this paper can be applied to the smart home system to realize the automatic adjustment and management of indoor air environment by integrating the functional modules such as air conditioner, air purifier and humidifier. This will improve the user's living comfort and health level.
- In the industrial environment, similar multi-objective optimization design can also be used for the design and optimization of the workshop air conditioning system to improve the comfort and safety of the working environment. For example, in high temperature, high humidity or polluted environments, the design of air conditioners, dehumidifiers and air purification equipment is optimized to ensure that the air quality in the work area meets the standard requirements.

References

- [1] Wang C ,Zhao Z ,Wang B , et al.Optimal fresh-air utilization strategy based on isocost Line: Adaptability analysis for constant temperature and humidity air-conditioning system[J]. Sustainable Energy Technologies and Assessments,2024,72104064-104064.
- [2] Mustufa A ,Anwar M ,Khan I T. Energy optimization of an institutional building to reduce the energy and air conditioning demand[J]. Journal of Building Engineering,2024,98111309-111309.
- [3] Zhang Y ,Zhao D ,Wu L , et al.Multi-objective optimization under the mixed utilization strategy of heat pump and electric drive waste heat of electric vehicles[J]. Applied Thermal Engineering,2025,259124900-124900.
- [4] Kim M ,Kwon S. Application of supervised and unsupervised learning for enhancing energy efficiency and thermal comfort in air conditioning scheduling under uncertain and dynamic environments[J]. Energy & Buildings,2024,325115028-115028.
- [5] Chen D ,Liu M ,Guo W , et al.Energy-efficient operation of portable air cleaners based on real-time prediction of non-uniform concentrations of indoor air pollutants in open offices[J]. Building and Environment,2024,256111478 -.
- [6] Yao C ,Lee S ,이승재 , et al.A Numerical Study for Optimization of Air Purifier Placement and Operation in School Classrooms[J]. The 2024,
- [7] Liu J ,Zhao L ,Hu K , et al.Optimization study of a closed-cycle low-temperature evaporation system based on mathematical model and CFD.[J]. Environmental research,2024,251(P2):118762-118762.
- [8] Hui Y ,Chengcheng W ,Yelong Z , et al.Experimental investigation of a thermochemical energy storage system based on MgSO₄-silica gel for building heating: adsorption/desorption performance testing and system optimization[J]. Energy Conversion and Management,2024,301118000 -.
- [9] Sankhyan S ,Clements N ,Heckman A , et al.Optimization of a Do-It-Yourself Air Cleaner Design to Reduce Residential Air Pollution Exposure for a Community Experiencing Environmental Injustices[J]. Atmosphere,2023,14(12):
- [10] Hooseung N ,Haneul C ,Haseong K , et al.Optimizing indoor air quality and noise levels in old school classrooms with air purifiers and HRV: A CONTAM simulation study[J]. Journal of Building Engineering,2023,73
- [11] Yongchao L ,Hao C ,Chihua L , et al.Multi-objective optimization design of a planar membrane humidifier based on NSGA-II and entropy weight TOPSIS[J]. Journal of Thermal Analysis and Calorimetry,2023,148(14):7147-7161.
- [12] Amit B ,Balaram K. Exergoeconomic assessment and optimization of a double effect absorption chiller integrated with a humidification-dehumidification desalination system[J]. Energy Conversion and Management,2021,247

Appendix

LIST OF MATERIALS

 1.1.1	Explore the influence of air conditioning location on air conditioning efficiency (air conditioning in the middle of the house)
 1.1.2	Explore the influence of air conditioning location on air conditioning efficiency (air conditioning location in the corner)
 1.1.3	Explore the influence of the number of air conditioners on the room temperature
 1.1.4	Exploring the influence of air outlet speed of air conditioning on room temperature
 1.1.5	Influence of air-conditioning air volume on room temperature
 1.2	The first question-to solve the best size and shape of air conditioning
 Q2	The second question is to solve the code.
 Q2-test	Second question test
 Q3	The third question to solve the code
 Q3-test	Third question test
 4.1	The fourth question to solve the code

Problem 1
The air conditioner is located in the middle of the house, exploring the influence of air volume on temperature
<pre>import numpy as np import matplotlib.pyplot as plt from matplotlib import cm # Set Chinese font support plt.rcParams['font.sans-serif'] = ['SimHei'] plt.rcParams['axes.unicode_minus'] = False #1. Define the room and physical parameters # Room size (m) room_length = 8.0# length room_width = 5.0# width room_height = 3.0# Height # Meshing nx = 30 # x-direction grid number, reduced to improve computational efficiency ny = 30# number of grids in y direction nz = 20# number of grids in z direction</pre>

```
dx = room_length / (nx-1)# Grid spacing (x direction)
dy = room_width / (ny-1)# grid spacing (y direction)
dz = room_height / (nz-1)# Grid spacing (z direction)

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order

# Time step and total time (set at 10 minutes)
dt = 0.1# sec, reduce time step to improve stability
total_time = 600# total time (10 minutes)
nt = int(total_time / dt) # time steps

# Thermal diffusivity (air)
alpha = thermal_diffusivity of 0.0001 m/s

# Initial temperature field (winter)
initial_temp_winter = 10.0# winter initial temperature
target_temp = 22.0# target temperature

# Outside ambient temperature
outside_temp_winter = 5.0# winter external temperature

#2. Define the shape and size of the air conditioner

def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air conditioner, the mask of the
    air conditioner area is generated.
    """
    mask = np.zeros((nx, ny, nz), dtype=bool)
    if shape == 'rectangle':
        # size = (width, height, depth)
        width, height, depth = size
        x_center, y_center, z_center = position

        # Calculate the start and end indexes
        x_start = int((x_center - width / 2) / dx)
        y_start = int((y_center - height / 2) / dy)
        z_start = int((z_center - depth / 2) / dz)
        x_end = int((x_center + width / 2) / dx)
```

```

y_end = int((y_center + height / 2) / dy)
z_end = int((z_center + depth / 2) / dz)

# Make sure the index is in the valid range
x_start = max(0, x_start)
y_start = max(0, y_start)
z_start = max(0, z_start)
x_end = min(nx, x_end)
y_end = min(ny, y_end)
z_end = min(nz, z_end)

mask[x_start:x_end, y_start:y_end, z_start:z_end] = True
return mask

#3. Establish a numerical simulation model of the temperature field (considering
the influence of wind speed, simplified to the relationship between air output
and temperature change speed)
def simulate_temperature(ac_shape, ac_size, initial_temp, outside_temp,
air_volume):
    """
    Simulate the temperature change under the given air conditioner shape and size,
    considering different air output.
    """
    T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
    T_new = T.copy() # Temporary variables are used to update the temperature
    field

    # Get the mask of the air conditioning area
    ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
room_width / 2, room_height / 2))

    # Adjust the temperature change speed according to the output air volume
    (simple assumption)
    temp_change_rate = air_volume * 0.01

    # Simulation process
    for n in range(nt):
        # Heat conduction calculation (three-dimensional heat conduction equation)
        T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + alpha * dt * (
            (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx **2 +
            (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy **2 +
            (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz **2
        )

```

```
# Boundary conditions (surrounding walls)
T_new[0, :, :] = outside_temp
T_new[-1, :, :] = outside_temp
T_new[:, 0, :] = outside_temp
T_new[:, -1, :] = outside_temp
T_new[:, :, 0] = outside_temp
T_new[:, :, -1] = outside_temp

# The temperature of the air-conditioning area is set to the target temperature
and adjusted according to the air output
if n * dt <= 600:
    ac_effect = min(1, n * dt * temp_change_rate)
    T_new[ac_mask] = initial_temp + ac_effect * (target_temp - initial_temp)

# Update temperature field
T = T_new.copy()

return T

#4. Simulate temperature changes in winter
ac_shapes = ['rectangle']
ac_sizes = {
    'rectangle': [(0.7, 0.5, 0.5)]
}
air_volumes = [1, 2, 3, 4, 5]

temperature_results = []
for air_volume in air_volumes:
    for shape in ac_shapes:
        sizes = ac_sizes[shape]
        for size in sizes:
            # Simulated winter
            T_winter = simulate_temperature(shape, size, initial_temp_winter,
                                             outside_temp_winter, air_volume)
            temperature_results.append(T_winter)

#5. Visualize the temperature distribution over time and extract the data on the
horizontal line of the air conditioner center
def plot_temperature_sequence(temperature_results, air_volumes):
```



```
plt.figure()
for I, t in enumerate(temperature_results):
    mid_z = nz // 2
    ac_center_y = ny // 2
    ac_center_x = np.arange(nx) * dx
    ac_center_temp = t[:, ac_center_y, mid_z]

    plt.plot(ac_center_x, ac_center_temp, label=f'discharge quantity of
    air: {air_volumes[ I ]}CMM')

plt.title('Winter Indoor Temperature Distribution under Different air Volume (10
minutes)')
plt.xlabel('X position (m)')
plt.ylabel('temperature (°C)')
plt.legend()
plt.show()

plot_temperature_sequence(temperature_results, air_volumes)
```

Problem 1

Air conditioning location in the middle of the house, to explore the impact of wind speed on temperature

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#1. Define the room and physical parameters

# Room size (m)
room_length = 8.0# length
room_width = 5.0# width
room_height = 3.0# Height
```

```

# Meshing
nx = 30 # x-direction grid number, reduced to improve computational efficiency
ny = 30 # number of grids in y direction
nz = 20 # number of grids in z direction
dx = room_length / (nx-1) # Grid spacing (x direction)
dy = room_width / (ny-1) # grid spacing (y direction)
dz = room_height / (nz-1) # Grid spacing (z direction)

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order

# Time Step and Total Time
dt = 0.1 # sec, reduce time step to improve stability
total_time = 600 # total time (10 minutes)
nt = int(total_time / dt) # time steps

# Thermal diffusivity (air)
alpha = thermal_diffusivity of 0.0001 m/s

# Initial temperature field (winter)
initial_temp_winter = 30.0 # winter initial temperature
target_temp = 26.0 # target temperature

# Outside ambient temperature
outside_temp_winter = 35.0 # winter external temperature

#2. Define the shape and size of the air conditioner

def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air conditioner, the mask of the
    air conditioner area is generated.
    """
    mask = np.zeros((nx, ny, nz), dtype=bool)
    if shape == 'rectangle':
        # size = (width, height, depth)
        width, height, depth = size
        x_center, y_center, z_center = position

        # Calculate the start and end indexes

```

```

x_start = int((x_center - width / 2) / dx)
y_start = int((y_center - height / 2) / dy)
z_start = int((z_center - depth / 2) / dz)
x_end = int((x_center + width / 2) / dx)
y_end = int((y_center + height / 2) / dy)
z_end = int((z_center + depth / 2) / dz)

# Make sure the index is in the valid range
x_start = max(0, x_start)
y_start = max(0, y_start)
z_start = max(0, z_start)
x_end = min(nx, x_end)
y_end = min(ny, y_end)
z_end = min(nz, z_end)

mask[x_start:x_end, y_start:y_end, z_start:z_end] = True
return mask

```

#3. Establish a numerical simulation model of the temperature field

```

def simulate_temperature(ac_shape, ac_size, initial_temp, outside_temp,
wind_speed):
    """

```

Simulate temperature changes for a given air conditioner shape and size.
Here it is simply assumed that wind speed affects heat exchange efficiency and is simulated by adjusting alpha.

```

    """
    adjusted_alpha = alpha * wind_speed
    T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
    T_new = T.copy() # Temporary variables are used to update the temperature field

```

```

    # Get the mask of the air conditioning area
    ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
room_width / 2, room_height / 2))

```

```

    # Simulate to 10 minutes (600 seconds)
    for n in range(nt):
        # Heat conduction calculation (three-dimensional heat conduction equation)
        T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + adjusted_alpha * dt * (
            (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx **2 +
            (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy **2 +
            (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz **2

```

```
)

# Boundary conditions (surrounding walls)
T_new[0, :, :] = outside_temp
T_new[-1, :, :] = outside_temp
T_new[:, 0, :] = outside_temp
T_new[:, -1, :] = outside_temp
T_new[:, :, 0] = outside_temp
T_new[:, :, -1] = outside_temp

# Set the air conditioning zone temperature to the target temperature
T_new[ac_mask] = target_temp

# Update temperature field
T = T_new.copy()

mid_z = nz // 2
ac_center_y = ny // 2
ac_center_x = np.arange(nx) * dx
ac_center_temp = T[:, ac_center_y, mid_z]

return ac_center_x, ac_center_temp

#4. Simulate temperature changes in winter
ac_shapes = ['rectangle']
ac_sizes = {
    'rectangle': [(0.7, 0.5, 0.5)]
}

wind_speeds = [1.0, 1.2, 1.5, 1.7, 2.0] # different wind speed conditions
for wind_speed in wind_speeds:
    for shape in ac_shapes:
        sizes = ac_sizes[shape]
        for size in sizes:
            # Simulated winter
            ac_center_x, ac_center_temp = simulate_temperature(shape, size,
                initial_temp_winter, outside_temp_winter, wind_speed)
            plt.plot(ac_center_x, ac_center_temp, label=f'air speed: {wind_speed}')
```

```
#5. Visualize the temperature distribution over time
plt.title('Indoor Temperature distribution under different wind speeds (10
minutes)')
plt.xlabel('X position (m)')
plt.ylabel('temperature (°C)')
plt.legend()
plt.show()
```

Problem 1

The air conditioner is located in the middle of the house, change the number of air conditioners

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#1. Define the room and physical parameters

# Room size (m)
room_length = 8.0# length
room_width = 5.0# width
room_height = 3.0# Height

# Meshing
nx = 30 # x-direction grid number, reduced to improve computational efficiency
ny = 30# number of grids in y direction
nz = 20# number of grids in z direction
dx = room_length / (nx-1)# Grid spacing (x direction)
dy = room_width / (ny-1)# grid spacing (y direction)
dz = room_height / (nz-1)# Grid spacing (z direction)

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order
```

```
# Time Step and Total Time
dt = 0.1# sec, reduce time step to improve stability
total_time = 600# total time (10 minutes)
nt = int(total_time / dt) # time steps

# Thermal diffusivity (air)
alpha = thermal diffusivity of 0.0001 m/s

# Initial temperature field (winter)
initial_temp_winter = 10.0# winter initial temperature
target_temp = 22.0# target temperature

# Outside ambient temperature
outside_temp_winter = 5.0# winter external temperature

#2. Define the shape and size of the air conditioner

def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air conditioner, the mask of the
    air conditioner area is generated.
    """
    mask = np.zeros((nx, ny, nz), dtype=bool)
    if shape == 'rectangle':
        # size = (width, height, depth)
        width, height, depth = size
        x_center, y_center, z_center = position

        # Calculate the start and end indexes
        x_start = int((x_center - width / 2) / dx)
        y_start = int((y_center - height / 2) / dy)
        z_start = int((z_center - depth / 2) / dz)
        x_end = int((x_center + width / 2) / dx)
        y_end = int((y_center + height / 2) / dy)
        z_end = int((z_center + depth / 2) / dz)

        # Make sure the index is in the valid range
        x_start = max(0, x_start)
        y_start = max(0, y_start)
        z_start = max(0, z_start)
        x_end = min(nx, x_end)
        y_end = min(ny, y_end)
        z_end = min(nz, z_end)
```

```
mask[x_start:x_end, y_start:y_end, z_start:z_end] = True
return mask
```

#3. Establish a numerical simulation model of the temperature field

```
def simulate_temperature(ac_num, ac_shape, ac_size, initial_temp,
    outside_temp):
```

```
    """
```

```
    Simulate temperature changes for a given air conditioner shape and size.
```

```
    """
```

```
    T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
```

```
    T_new = T.copy() # Temporary variables are used to update the temperature
    field
```

```
    # Determine the location of multiple air conditioners
```

```
    ac_positions = [(room_length * (2 * I + 1) / (2 * ac_num), room_width / 2,
    room_height / 2) for I in range(ac_num)]
```

```
    # Get the mask of the air conditioning area
```

```
    ac_masks = []
```

```
    for pos in ac_positions:
```

```
        ac_mask = get_ac_mask(ac_shape, ac_size, pos)
```

```
    ac_masks.append(ac_mask)
```

```
    # Simulation process
```

```
    for n in range(nt):
```

```
        # Heat conduction calculation (three-dimensional heat conduction equation)
```

```
        T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + alpha * dt * (
            (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx **2 +
            (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy **2 +
            (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz **2
        )
```

```
    # Boundary conditions (surrounding walls)
```

```
    T_new[0, :, :] = outside_temp
```

```
    T_new[-1, :, :] = outside_temp
```

```
    T_new[:, 0, :] = outside_temp
```

```
    T_new[:, -1, :] = outside_temp
```

```
    T_new[:, :, 0] = outside_temp
```

```
    T_new[:, :, -1] = outside_temp
```

```
    # Set the air conditioning zone temperature to the target temperature
```

```
for ac_mask in ac_masks:
    T_new[ac_mask] = target_temp
```

```
# Update temperature field
T = T_new.copy()
```

```
return T
```

```
#4. Simulate the temperature change in winter under different number of air conditioners and plot
```

```
ac_shapes = ['rectangle']
ac_sizes = {
    'rectangle': [(0.7, 0.5, 0.5)]
}
```

```
ac_num_list = [1, 2, 3, 4, 5]
```

```
for ac_num in ac_num_list:
    for shape in ac_shapes:
        sizes = ac_sizes[shape]
        for size in sizes:
            # Simulated winter
            T_winter = simulate_temperature(ac_num, shape, size, initial_temp_winter,
                                             outside_temp_winter)
```

```
# Visualize the change of temperature distribution over time and extract the data
on the horizontal line of the air conditioner center
```

```
mid_z = nz // 2
ac_center_y = ny // 2
ac_center_x = np.arange(nx) * dx
ac_center_temp = T_winter[:, ac_center_y, mid_z]
```

```
plt.plot(ac_center_x, ac_center_temp, label = f'{ac_num} air conditioners')
```

```
plt.title('Indoor temperature distribution under different number of air
conditioners (winter, 10 min)')
plt.xlabel('X position (m)')
plt.ylabel('temperature (°C)')
plt.legend()
plt.show()
```


Problem 1**Air conditioning in the corner**

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#1. Define the room and physical parameters

# Room size (m)
room_length = 8.0# length
room_width = 5.0# width
room_height = 3.0# Height

# Meshing
nx = 30 # x-direction grid number, reduced to improve computational efficiency
ny = 30# number of grids in y direction
nz = 20# number of grids in z direction
dx = room_length / (nx-1)# Grid spacing (x direction)
dy = room_width / (ny-1)# grid spacing (y direction)
dz = room_height / (nz-1)# Grid spacing (z direction)

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order

# Time Step and Total Time
dt = 0.1# sec, reduce time step to improve stability
total_time = 600# total time (10 minutes)
nt = int(total_time / dt) # time steps

# Thermal diffusivity (air)
alpha = thermal diffusivity of 0.0001 m/s

# Initial temperature field (winter)
initial_temp_winter = 10.0# winter initial temperature
```

```
target_temp = 22.0# target temperature
```

```
# Outside ambient temperature
```

```
outside_temp_winter = 5.0# winter external temperature
```

```
#2. Define the shape and size of the air conditioner
```

```
def get_ac_mask(shape, size, position):
```

```
"""
```

```
According to the shape, size and position of the air conditioner, the mask of the  
air conditioner area is generated.
```

```
"""
```

```
mask = np.zeros((nx, ny, nz), dtype=bool)
```

```
if shape == 'rectangle':
```

```
# size = (width, height, depth)
```

```
width, height, depth = size
```

```
x_center, y_center, z_center = position
```

```
# Calculate the start and end indexes
```

```
x_start = int((x_center - width / 2) / dx)
```

```
y_start = int((y_center - height / 2) / dy)
```

```
z_start = int((z_center - depth / 2) / dz)
```

```
x_end = int((x_center + width / 2) / dx)
```

```
y_end = int((y_center + height / 2) / dy)
```

```
z_end = int((z_center + depth / 2) / dz)
```

```
# Make sure the index is in the valid range
```

```
x_start = max(0, x_start)
```

```
y_start = max(0, y_start)
```

```
z_start = max(0, z_start)
```

```
x_end = min(nx, x_end)
```

```
y_end = min(ny, y_end)
```

```
z_end = min(nz, z_end)
```

```
mask[x_start:x_end, y_start:y_end, z_start:z_end] = True
```

```
return mask
```

```
#3. Establish a numerical simulation model of the temperature field
```

```
def simulate_temperature(ac_shape, ac_size, initial_temp, outside_temp):
```

```
T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
```

```
T_new = T.copy() # Temporary variables are used to update the temperature
```

field

```
# Change the air conditioner position to the room corner, here select (0,0, 0)
near the coordinates (dx, dy, dz)
```

```
ac_mask = get_ac_mask(ac_shape, ac_size, position=(dx, dy, dz))
```

```
selected_times = [0, 60, 120, 180, 240, 300, 360, 420, 480, 540, 600]
```

```
selected_temperature = []
```

```
# Simulation process
```

```
for n in range(nt):
```

```
# Heat conduction calculation (three-dimensional heat conduction equation)
```

```
T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + alpha * dt * (
    (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx **2 +
    (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy **2 +
    (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz **2
)
```

```
# Boundary conditions (surrounding walls)
```

```
T_new[0, :, :] = outside_temp
```

```
T_new[-1, :, :] = outside_temp
```

```
T_new[:, 0, :] = outside_temp
```

```
T_new[:, -1, :] = outside_temp
```

```
T_new[:, :, 0] = outside_temp
```

```
T_new[:, :, -1] = outside_temp
```

```
# Set the air conditioning zone temperature to the target temperature
```

```
T_new[ac_mask] = target_temp
```

```
# Update temperature field
```

```
T = T_new.copy()
```

```
if n * dt in selected_times:
```

```
    selected_temperature.append(T.copy())
```

```
return selected_temperature
```

```
#4. Simulate temperature changes in winter
```

```
ac_shapes = ['rectangle']
```

```
ac_sizes = {
```

```
    'rectangle': [(0.7, 0.5, 0.5)]
```

```
}
```

```
for shape in ac_shapes:
    sizes = ac_sizes[shape]
    for size in sizes:
        # Simulated winter
        T_winter = simulate_temperature(shape, size, initial_temp_winter,
                                         outside_temp_winter)
```

#5. Visualize the temperature distribution over time and extract the data on the horizontal line of the air conditioner center

```
def plot_temperature_sequence(temperature_sequence, title):
```

```
    num_time_steps = len(temperature_sequence)
```

```
    plt.figure()
```

```
    for I, t in enumerate(temperature_sequence):
```

```
        mid_z = nz // 2
```

```
        ac_center_y = ny // 2
```

```
        ac_center_x = np.arange(nx) * dx
```

```
        ac_center_temp = t[:, ac_center_y, mid_z]
```

```
    plt.plot(ac_center_x, ac_center_temp, label=f'{int(I * 60 / 60)+1}min')
```

```
    # Extract diagonal temperature
```

```
    diag_x = np.linspace(0, room_length, nx)
```

```
    diag_y = np.linspace(0, room_width, ny)
```

```
    diag_z = np.linspace(0, room_height, nz)
```

```
    diag_temp = []
```

```
    for j in range(nx):
```

```
        x_index = j
```

```
        y_index = int(j * (ny - 1) / (nx - 1))
```

```
        z_index = int(j * (nz - 1) / (nx - 1))
```

```
        diag_temp.append(t[x_index, y_index, z_index])
```

```
    plt.plot(diag_x, diag_temp, label=f'{int(I * 60 / 60)+1}min')
```

```
    plt.title(title)
```

```
    plt.xlabel('X position (m)')
```

```
    plt.ylabel('temperature (°C)')
```

```
    plt.legend()
```

```
plt.show()
```

```
plot_temperature_sequence(T_winter, 'Indoor temperature distribution in  
winter (diagonal)')
```

Problem 1

Air conditioning in the middle of the house

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#1. Define the room and physical parameters

# Room size (m)
room_length = 8.0# length
room_width = 5.0# width
room_height = 3.0# Height

# Meshing
nx = 30 # x-direction grid number, reduced to improve computational efficiency
ny = 30# number of grids in y direction
nz = 20# number of grids in z direction
dx = room_length / (nx-1)# Grid spacing (x direction)
dy = room_width / (ny-1)# grid spacing (y direction)
dz = room_height / (nz-1)# Grid spacing (z direction)

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order

# Time Step and Total Time
dt = 0.1# sec, reduce time step to improve stability
total_time = 600# total time (10 minutes)
nt = int(total_time / dt) # time steps
```

```
# Thermal diffusivity (air)
alpha = thermal diffusivity of 0.0001 m/s

# Initial temperature field (winter)
initial_temp_winter = 10.0# winter initial temperature
target_temp = 22.0# target temperature

# Outside ambient temperature
outside_temp_winter = 5.0# winter external temperature

#2. Define the shape and size of the air conditioner

def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air conditioner, the mask of the
    air conditioner area is generated.
    """
    mask = np.zeros((nx, ny, nz), dtype=bool)
    if shape == 'rectangle':
        # size = (width, height, depth)
        width, height, depth = size
        x_center, y_center, z_center = position

        # Calculate the start and end indexes
        x_start = int((x_center - width / 2) / dx)
        y_start = int((y_center - height / 2) / dy)
        z_start = int((z_center - depth / 2) / dz)
        x_end = int((x_center + width / 2) / dx)
        y_end = int((y_center + height / 2) / dy)
        z_end = int((z_center + depth / 2) / dz)

        # Make sure the index is in the valid range
        x_start = max(0, x_start)
        y_start = max(0, y_start)
        z_start = max(0, z_start)
        x_end = min(nx, x_end)
        y_end = min(ny, y_end)
        z_end = min(nz, z_end)

        mask[x_start:x_end, y_start:y_end, z_start:z_end] = True
    return mask
```

#3. Establish a numerical simulation model of the temperature field

```
def simulate_temperature(ac_shape, ac_size, initial_temp, outside_temp):
    """
    Simulate temperature changes for a given air conditioner shape and size.
    """
    T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
    T_new = T.copy() # Temporary variables are used to update the temperature
    field

    # Get the mask of the air conditioning area
    ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
    room_width / 2, room_height / 2))

    selected_times = [0, 60, 120, 180, 240, 300, 360, 420, 480, 540, 600]
    selected_temperature = []

    # Simulation process
    for n in range(nt):
        # Heat conduction calculation (three-dimensional heat conduction equation)
        T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + alpha * dt * (
            (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx **2 +
            (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy **2 +
            (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz **2
        )

        # Boundary conditions (surrounding walls)
        T_new[0, :, :] = outside_temp
        T_new[-1, :, :] = outside_temp
        T_new[:, 0, :] = outside_temp
        T_new[:, -1, :] = outside_temp
        T_new[:, :, 0] = outside_temp
        T_new[:, :, -1] = outside_temp

        # Set the air conditioning zone temperature to the target temperature
        T_new[ac_mask] = target_temp

        # Update temperature field
        T = T_new.copy()

        if n * dt in selected_times:
            selected_temperature.append(T.copy())

    return selected_temperature
```

#4. Simulate temperature changes in winter

```
ac_shapes = ['rectangle']
ac_sizes = {
    'rectangle': [(0.7, 0.5, 0.5)]
}
```

```
for shape in ac_shapes:
```

```
    sizes = ac_sizes[shape]
```

```
    for size in sizes:
```

```
        # Simulated winter
```

```
        T_winter = simulate_temperature(shape, size, initial_temp_winter,
                                         outside_temp_winter)
```

#5. Visualize the temperature distribution over time and extract the data on the horizontal line of the air conditioner center

```
def plot_temperature_sequence(temperature_sequence, title):
```

```
    num_time_steps = len(temperature_sequence)
```

```
    plt.figure()
```

```
    for I, t in enumerate(temperature_sequence):
```

```
        mid_z = nz // 2
```

```
        ac_center_y = ny // 2
```

```
        ac_center_x = np.arange(nx) * dx
```

```
        ac_center_temp = t[:, ac_center_y, mid_z]
```

```
    plt.plot(ac_center_x, ac_center_temp, label=f'{int(I * 60 / 60)+1}min')
```

```
    plt.title(title)
```

```
    plt.xlabel('X position (m)')
```

```
    plt.ylabel('temperature (°C)')
```

```
    plt.legend()
```

```
    plt.show()
```

```
plot_temperature_sequence(T_winter, 'Indoor temperature distribution in
winter')
```


Problem 1

Solve

```

import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
from scipy.optimize import minimize
from mpl_toolkits.mplot3d.art3d import Poly3DCollection

# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

#1. Define the room and physical parameters

# Room size (m)
room_length = 8.0# length
room_width = 5.0# width
room_height = 3.0# Height

# Meshing
nx = 30 # x-direction grid number, reduced to improve computational efficiency
ny = 30# number of grids in y direction
nz = 20# number of grids in z direction
dx = room_length / (nx-1)# Grid spacing (x direction)
dy = room_width / (ny-1)# grid spacing (y direction)
dz = room_height / (nz-1)# grid spacing (z direction})

x = np.linspace(0, room_length, nx) # x coordinate
y = np.linspace(0, room_width, ny) # y coordinate
z = np.linspace(0, room_height, nz) # z coordinate

X, Y, Z = np.meshgrid(x, y, z, indexing = 'ij') # Ensure consistent index order

# Time Step and Total Time
dt = 0.1# sec, reduce time step to improve stability
total_time = 600# total time (10 minutes)
nt = int(total_time / dt) # time steps

# Thermal diffusivity (air)
alpha = #0.0001 thermal diffusivity, m ^ 2/s

# Initial temperature field (summer and winter)
initial_temp_summer = 30.0# Summer Initial Temperature
initial_temp_winter = 10.0# winter initial temperature

```

```
target_temp = 22.0# target temperature

# Outside ambient temperature
outside_temp_summer = 35.0# summer outside temperature
outside_temp_winter = 5.0# winter external temperature

#2. Define different shapes and sizes of air conditioners

def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air conditioner, the mask of the
    air conditioner area is generated.
    """
    mask = np.zeros((nx, ny, nz), dtype=bool)
    if shape == 'rectangle ':
        # size = (width, height, depth)
        width, height, depth = size
        x_center, y_center, z_center = position

        # Calculate the start and end indexes
        x_start = int((x_center - width / 2) / dx)
        y_start = int((y_center - height / 2) / dy)
        z_start = int((z_center - depth / 2) / dz)
        x_end = int((x_center + width / 2) / dx)
        y_end = int((y_center + height / 2) / dy)
        z_end = int((z_center + depth / 2) / dz)

        # Make sure the index is in the valid range
        x_start = max(0, x_start)
        y_start = max(0, y_start)
        z_start = max(0, z_start)
        x_end = min(nx, x_end)
        y_end = min(ny, y_end)
        z_end = min(nz, z_end)

        mask[x_start:x_end, y_start:y_end, z_start:z_end] = True

    elif shape == 'circle ':
        # size = radius
        radius = size
        cx = int(position[0] / dx)
        cy = int(position[1] / dy)
        cz = int(position[2] / dz)
```

```

for I in range(nx):
    for j in range(ny):
        for k in range(nz):
            distance = ((I - cx) * dx) ** 2 + ((j - cy) * dy) ** 2 + ((k - cz) * dz) ** 2
            if distance <= radius ** 2:
                mask[I, j, k] = True
        return mask

```

#3. Establish a numerical simulation model of the temperature field

```

def simulate_temperature(ac_shape, ac_size, initial_temp, outside_temp):
    """
    Simulate temperature changes for a given air conditioner shape and size.
    """
    T = np.full((nx, ny, nz), initial_temp) # Initialize the temperature field
    T_new = T.copy() # Temporary variables are used to update the temperature field

    # Get the mask of the air conditioning area
    ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
                                                         room_width / 2, room_height / 2))

    # Simulation process
    for n in range(nt):
        # Heat conduction calculation (three-dimensional heat conduction equation)
        T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] + alpha * dt * (
            (T[2:, 1:-1, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]) / dx ** 2 +
            (T[1:-1, 2:, 1:-1] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, :-2, 1:-1]) / dy ** 2 +
            (T[1:-1, 1:-1, 2:] - 2 * T[1:-1, 1:-1, 1:-1] + T[1:-1, 1:-1, :-2]) / dz ** 2
        )

        # Boundary conditions (surrounding walls)
        T_new[0, :, :] = outside_temp
        T_new[-1, :, :] = outside_temp
        T_new[:, 0, :] = outside_temp
        T_new[:, -1, :] = outside_temp
        T_new[:, :, 0] = outside_temp
        T_new[:, :, -1] = outside_temp

        # Set the air conditioning zone temperature to the target temperature
        T_new[ac_mask] = target_temp

    # Update temperature field
    T = T_new.copy()

```

```
return T

#4. Simulate temperature changes in summer and winter
# Define the shape and size list of air conditioners
ac_shapes = ['rectangle', 'circle']
ac_sizes = {
    'rectangle': [], # Rectangular air conditioners of different sizes (width, height,
    depth)
    'circle': [0.5, 0.7, 1.0] # Spherical air conditioners with different radii
}
widths = np.arange(0.5, 2.1, 0.1)
heights = np.arange(0.5, 1.6, 0.1)
depths = np.arange(0.5, 1.1, 0.1)
for width in widths:
    for height in heights:
        for depth in depths:
            ac_sizes['rectangle'].append((width, height, depth))
# Store results
results = []

for shape in ac_shapes:
    sizes = ac_sizes[shape]
    for size in sizes:
        # Simulated Summer
        T_summer = simulate_temperature(shape, size, initial_temp_summer,
        outside_temp_summer)
        # Calculate summer temperature variance
        variance_summer = np.var(T_summer - target_temp)

        # Simulated winter
        T_winter = simulate_temperature(shape, size, initial_temp_winter,
        outside_temp_winter)
        # Calculate winter temperature variance
        variance_winter = np.var(T_winter - target_temp)

        # Total variance
        total_variance = variance_summer + variance_winter

        # Save results
        results.append ({
            'shape': shape
            'size': size
            'variance_summer': variance_summer,
```

```
'variance_winter': variance_winter,
'total_variance': total_variance
})

print(f "Shape: {shape}, Dimension: {size}, Total Temperature Variance:
{total_variance:.2f}")

#5. Compare the performance of different air conditioning designs and choose
the best solution

# Find the solution with the smallest total temperature variance
best_result = min(results, key=lambda x: x['total_variance'])

print("\nThe best air conditioning design:")
print(f "Shape: {best_result['shape']}")
print(f "size: {best_result['size']}")
print(f "summer temperature variance: {best_result['variance_summer']:.2f}")
print(f "winter temperature variance: {best_result['variance_winter']:.2f}")
print(f "total temperature variance: {best_result['total_variance']:.2f}")

#6. Visualize the temperature distribution of the best solution

def plot_temperature(T, title):
    """
    A two-dimensional section of the intermediate layer of the three-dimensional
    temperature distribution is plotted.
    """
    plt.figure(figsize=(8, 6))
    mid_z = nz // 2
    plt.contourf(X[:, :, mid_z], Y[:, :, mid_z], T[:, :, mid_z], 20, cmap=cm.jet)
    plt.colorbar(label = 'Temperature (°C)')
    plt.title(title)
    plt.xlabel('X position (m)')
    plt.ylabel('Y position (m)')

# Draw air conditioning area boundary
ac_shape = best_result['shape']
ac_size = best_result['size']
ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
room_width / 2, room_height / 2))
ac_slice = ac_mask[:, :, mid_z]
plt.contour(X[:, :, mid_z], Y[:, :, mid_z], ac_slice.astype(int), levels=[0.5],
colors='white', linewidths=2)
```

```
plt.show()
```

```
# Simulate and plot the temperature distribution for the best scenario (summer)
T_summer_best = simulate_temperature(best_result['shape'], best_result['size'],
initial_temp_summer,
outside_temp_summer)
plot_temperature(T_summer_best, 'Temperature distribution for optimal air-
conditioning design in summer')
```

```
# Simulate and plot the temperature distribution of the best solution (winter)
T_winter_best = simulate_temperature(best_result['shape'], best_result['size'],
initial_temp_winter,
outside_temp_winter)
plot_temperature(T_winter_best, 'Temperature distribution for optimal air-
conditioning design in winter')
```

Problem 2

problem solving

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
```

```
# Set Chinese font support
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
```

```
#1. Room parameter setting (given size according to topic)
room_length = 8.0# Room Length 8 m
room_width = 5.0# room width 5 m
room_height = 3.0# room height 3 m (modified according to the title)
```

```
# Meshing
nx = 50 # x direction grid number
ny = 50# number of grids in y direction
nz = 30 # z direction grid number (new)
dx = room_length / (nx - 1)
dy = room_width / (ny - 1)
dz = room_height / (nz - 1)
x = np.linspace(0, room_length, nx)
y = np.linspace(0, room_width, ny)
```

```

z = np.linspace(0, room_height, nz)
X, Y, Z = np.meshgrid(x, y, z)

# Different shapes and sizes of air purifiers
def get_ac_mask(shape, size, position):
    """
    According to the shape, size and position of the air purifier, the mask of the
    cleaning area is generated.
    """
    mask = np.zeros((nz, ny, nx), dtype=bool)
    if shape == 'rectangle':
        width, height, depth = size# Added depth (for rectangular air conditioners)
        x_start = int(position[0] / dx)
        y_start = int(position[1] / dy)
        z_start = int(position[2] / dz)
        x_end = int((position[0] + width) / dx)
        y_end = int((position[1] + height) / dy)
        z_end = int((position[2] + depth) / dz)
        mask[z_start:z_end, y_start:y_end, x_start:x_end] = True
    elif shape == 'circle':
        radius = size
        cx = int(position[0] / dx)
        cy = int(position[1] / dy)
        cz = int(position[2] / dz)
        for I in range(nz):
            for j in range(ny):
                for k in range(nx):
                    if (I-cz)**2 + (j - cy)**2 + (k - cx)**2 <= (radius / dx)**2:
                        mask[ I, j, k] = True
    return mask

# Air flow and pollutant diffusion coefficient
alpha = 0.1

# Simulation of air purification effect
def simulate_purification(ac_shape, ac_size, pollution_concentration):
    """
    Simulate the purification effect of air purifiers in different shapes and sizes.
    """
    T = np.copy(pollution_concentration) # initial pollution concentration field
    T_new = T.copy() # Temporary variable used to update the concentration field

```

```

# Get the mask of the air purifier area
ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
room_width / 2, room_height / 2))

# Simulation of the purification process
for _ in range(100): #100-step iteration
# Assuming that the air purifier can reduce the concentration of pollutants in its
area to zero
T_new[ac_mask] = 0.0

# Diffusion of pollutant concentration in other areas (simulation of air flow and
pollutant dispersion)
T_new[1:-1, 1:-1, 1:-1] = T[1:-1, 1:-1, 1:-1] * 0.9 + alpha * (
T[1:-1, 1:-1, 2:] + T[1:-1, 1:-1, :-2] +
T[1:-1, 1:-1, 2:] + T[1:-1, 1:-1, :-2] +
T[2:, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]-
6 * T[1:-1, 1:-1, 1:-1]) # three-dimensional diffusion formula

T = T_new.copy()

return T

#2. Define the objective function: minimize the total variance of the pollutant
concentration after purification.
def objective(params):
ac_shape = 'rectangle'
ac_size = (params[0], params[1], params[2])
pollution_concentration = np.random.uniform(0.5, 1.0, (nz, ny, nx))

# Simulated purification effect
T = simulate_purification(ac_shape, ac_size, pollution_concentration)

# Calculate the total variance of pollutant concentration after purification
return np.var(T)

# Calculate the numerical gradient
def numerical_gradient(func, x, h=1e-5):
grad = np.zeros_like(x)
for i in range(len(x)):
x_plus = x.copy()
x_plus[ i ] += h

```



```

x_minus = x.copy()
x_minus[ I] -= h
grad[ I] = (func(x_plus) - func(x_minus)) / (2 * h)
return grad

# gradient descent method
def gradient_descent(func, initial_guess, learning_rate=0.01,
num_iterations=1000):
x = np.array(initial_guess)
for _ in range(num_iterations):
grad = numerical_gradient(func, x)
x = x - learning_rate * grad
# Add size range limits
x = np.clip(x, [0, 0, 0], [room_length - 0.001, room_width - 0.001, room_height
- 0.001])
return x

Initial guess: Air purifier size (length, width, height)
initial_guess = [1,1, 1]

# Perform gradient descent optimization
best_ac_size = gradient_descent(objective, initial_guess)

# Output the best result
print(f "optimal air purifier size: length = {best_ac_size[0]:.2f} m, width =
{best_ac_size[1]:.2f} m, height = {best_ac_size[2]:.2f} m")

#4. Calculate the purification effect at the optimal size
best_purification = simulate_purification('rectangle', (best_ac_size[0],
best_ac_size[1], best_ac_size[2]),
np.random.uniform(0.5, 1.0, (nz, ny, nx)))

# Visualize the purification effect (select the appropriate slice display)
def plot_purification(T, title):
plt.figure(figsize=(6, 5))
Plt. contourf(X[:, :, 0], Y[:, :, 0], T[nz // 2, :, :], 20, cmap = cm.jet) # Select the
slice in the middle of the z axis to draw
plt.colorbar(label='pollution concentration')

```

```
plt.xlabel('X positon (m)')
plt.ylabel('Y positon (m)')
plt.show()
```

```
#5. Visualize the purification effect under the best design
plot_purification(best_purification, 'Purification effect of the best air purifier
design')
```

Problem 2

robustness test

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm

#1. Room parameter setting (given size according to topic)
room_length = 8.0# Room Length 8 m
room_width = 5.0# room width 5 m
room_height = 3.0# room height 3 m (modified according to the title)

# Meshing
nx = 50 # x direction grid number
ny = 50# number of grids in y direction
nz = 30 # z direction grid number (new)
dx = room_length / (nx - 1)
dy = room_width / (ny - 1)
dz = room_height / (nz - 1)
x = np.linspace(0, room_length, nx)
y = np.linspace(0, room_width, ny)
z = np.linspace(0, room_height, nz)
X, Y, Z = np.meshgrid(x, y, z)

# Different shapes and sizes of air purifiers
def get_ac_mask(shape, size, position):
    mask = np.zeros((nz, ny, nx), dtype=bool)
    if shape == 'rectangle ':
        width, height, depth = size
        x_start = int(position[0] / dx)
        y_start = int(position[1] / dy)
        z_start = int(position[2] / dz)
```

```

x_end = int((position[0] + width) / dx)
y_end = int((position[1] + height) / dy)
z_end = int((position[2] + depth) / dz)
mask[z_start:z_end, y_start:y_end, x_start:x_end] = True
elif shape == 'circle ':
    radius = size
    cx = int(position[0] / dx)
    cy = int(position[1] / dy)
    cz = int(position[2] / dz)
    for I in range(nz):
        for j in range(ny):
            for k in range(nx):
                dist = (I-cz) **2 + (j - cy)** 2 + (k - cx) **2
                if dist <= (radius / dx) **2:
                    mask[ I, j, k] = True
    return mask

# Air flow and pollutant diffusion coefficient
alpha = 0.1

# Simulation of air purification effect
def simulate_purification(ac_shape, ac_size, pollution_concentration):
    T = np.copy(pollution_concentration)
    T_new = T.copy()

    # Get the mask of the air purifier area
    ac_mask = get_ac_mask(ac_shape, ac_size, position=(room_length / 2,
room_width / 2, room_height / 2))

    # Simulation of the purification process
    for _ in range(100):
        # Assuming that the air purifier can reduce the concentration of pollutants in its
        area to zero
        T_new[ac_mask] = 0.0

    # Diffusion of pollutant concentration in other areas (simulation of air flow and
    pollutant dispersion)
    T_new[1:-1,1:-1,1:-1] = (
    T[1:-1, 1:-1, 1:-1] * 0.9 + alpha * (
    T[1:-1, 1:-1, 2:] + T[1:-1, 1:-1, :-2] +
    T[1:-1, 2:, 1:-1] + T[1:-1, :-2, 1:-1] +
    T[2:, 1:-1, 1:-1] + T[:-2, 1:-1, 1:-1]-

```

```
6 * T[1:-1, 1:-1, 1:-1]))
T = T_new.copy()

return T

# Optimized air purifier size
best_ac_size = np.array([0.82, 0.92, 1.18])

# Model inspection part
def model_test():
    # Change initial pollution concentration to normal distribution
    pollution_concentration = np.random.normal(loc=0.5, scale=0.1, size=(nz, ny,
nx))

    # Draw initial pollution concentration distribution map
    def plot_initial_pollution(T, title):
        plt.figure(figsize=(6, 5))
        plt.contourf(X[:, :, 0], Y[:, :, 0], T[nz // 2, :, :], 20, cmap=cm.jet)
        plt.colorbar(label='Pollution Concentration')
        plt.title(title)
        plt.xlabel('X position(m)')
        plt.ylabel('Y position(m)')
        plt.show()

    plot_initial_pollution(pollution_concentration, 'Initial Pollution Concentration
Distribution')

    # Calculate the purification effect under the optimal size
    best_purification = simulate_purification('rectangle', (best_ac_size[0],
best_ac_size[1], best_ac_size[2]),
pollution_concentration)

    # Visualize the purification effect (select the appropriate slice display)
    def plot_purification(T, title):
        plt.figure(figsize=(6, 5))
        plt.contourf(X[:, :, 0], Y[:, :, 0], T[nz // 2, :, :], 20, cmap=cm.jet)
        plt.colorbar(label='Pollution Concentration')
        plt.title(title)
        plt.xlabel('X position(m)')
        plt.ylabel('Y position(m)')
        plt.show()
```

```
# Visualize inspection results
plot_purification(best_purification, 'Purification effect of optimal air purifier
design under normal distribution initial concentration')

if __name__ == '__main__':
    model_test()
```

Problem 3

problem solving

```
import numpy as np
from scipy.optimize import minimize
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif']=['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# Constant setting
k = #0.02 evaporation coefficient, unit: m/s
V_max = #2 maximum volume, unit: m³
max_power = #200 maximum power, unit: W
max_noise = #50 maximum noise, unit: dB
position_factor = 1.2# location factor, assuming the air purifier is located in the
center of the room

# Initial guess value (shape type, size parameter 1, size parameter 2, size
parameter 3, airflow speed, humidification time)
initial_values = [0, 0.05, 0.2, 0.5, 1.0, 12.0]

# Hypothetical function of power consumption and noise
def power_consumption(flow_speed):
    """ "Calculate power consumption based on airflow velocity (assuming
    proportional to the cube of airflow velocity)" """
    return 0.5 * flow_speed **3+50# power consumption is proportional to the cube
of airflow speed

def noise_level(flow_speed):
    """ "Calculate noise based on airflow velocity (assuming linear relationship with
    airflow velocity)" """
    return 40+2 * flow_speed# assumes a linear relationship between noise and
airflow velocity

# Objective function of humidification efficiency
def humidification_efficiency(params):
```

```

"""
Humidification efficiency is calculated.
params[0] = parameter 1 (the size of the corresponding shape, such as radius,
length, etc.)
params[1] = parameter 2 (corresponding to airflow velocity)
params[2] = parameter 3 (corresponding to humidification time)
"""
shape_type = int(params[0]) # shape type (0 = cube, 1= column, 2 = box)
size_params = params[1:4] # The parameters of the corresponding shape, such
as the length, width, and height of the cuboid
v = params[4] # Airflow velocity
t = params[5] # humidification time

# Calculation of water surface area
if shape_type == 0: # cube
A = size_params[0] **2 # water surface area is the square of the side length
elif shape_type == 1: # Cylindrical
r = size_params[0] # radius
A = np.pi * r **2 # water surface area is sphere area
else: # cuboid
A = size_params[0] * size_params[1] # The water surface area is length
multiplied by width

Calculate humidification efficiency: efficiency = k * A * v * t * position factor
efficiency = k * A * v * t * position_factor

# Calculate power and noise
power = power_consumption(v)
noise = noise_level(v)

# Penalty if power or noise exceeds limits
if power > max_power:
efficiency -= (power-max_power) * 10 # Penalty: the part that exceeds the power
limit
if noise > max_noise:
efficiency -= (noise-max_noise) * 5 # Penalty: Exceeding the noise limit

return -efficiency # for maximum humidification efficiency

# Volume Constraint: Volume calculation of different shapes
def volume_constraint(params):
shape_type = int(params[0]) # shape type (0 = cube, 1= column, 2 = box)
size_params = params[1:4] # Size parameter of the corresponding shape
height = size_params[-1] # height parameter

```

```

if shape_type == 0: # cube
    volume = size_params[0] **3
elif shape_type == 1: # Cylindrical
    r = size_params[0] # radius
    volume = np.pi * r **2 * height# volume = circle area * height
else: # cuboid
    volume = size_params[0] * size_params[1] * height# volume = length * width
    * height

return V_max-volume# The volume cannot exceed the maximum volume

# Constraint list
constraints = [{'type': 'ineq', 'fun': volume_constraint}] # Volume Constraint

# Boundary conditions: the value range of water surface area, airflow velocity
and humidification time
bounds = [
    Shape type: (0,2), # = 0 cube, =1 cylinder, = 2 cuboid
    (0.01, 3.0), # radius or length, width, side length range
    (0.01, 3.0), # the second dimension of the box or the radius of the column
    (0.01, 3.0), # range of heights
    (0.1, 2.0), # Airflow Velocity Range
    (1,24)# Humidification time range (hours)
]

# Call the SciPy optimization function to optimize
result = minimize(humidification_efficiency, initial_values, bounds=bounds,
constraints=constraints)

# Output optimization results
optimal_shape = result.x[0] # Optimized shape type
optimal_size_params = result.x[1:4] # Optimized size parameter
optimal_v = result.x[4] # Optimized airflow velocity
optimal_t = result.x[5] # Optimized humidification time
optimal_efficiency = -result.fun# Maximum humidification efficiency

print(f"Optimized shape type: {'Cube', 'Cyland', 'Cuboid'
[int(optimal_shape)]}")
print(f"optimized size parameter: {optimal_size_params}")
print(f"optimized airflow velocity: {optimal_v:.2f} m/s")
print(f"optimized humidification time: {optimal_t:.2f} hours")
print(f"Maximum humidification efficiency: {optimal_efficiency:.2f}")

```

```

# Visualization: Draw the relationship between the water surface area of the best
shape and the airflow speed according to the optimization results
shape_names = ['Cube', 'Cylinder', 'Cuboid ']
A_vals = np.linspace(0.01, 1.0, 20)
v_vals = np.linspace(0.1, 2.0, 20)

efficiency_vals = np.zeros((len(A_vals), len(v_vals)))
for I, A in enumerate(A_vals):
    for j, v in enumerate(v_vals):
        efficiency_vals [I, j] = k * A * v * optimal_t * position_factor# using the
        optimized humidification time and location factor

# Draw a heat map of humidification efficiency
A_mesh, V_mesh = np.meshgrid(A_vals, v_vals)
plt.figure(figsize=(8, 6))
plt.contourf(A_mesh, V_mesh, efficiency_vals, levels=50, cmap='viridis')
plt.colorbar(label='Efficiency of humidification')
plt.xlabel('room area (stere)')
plt.ylabel('airflow velocity (m/s)')
plt.title('Humidification efficiency -room area vs airflow speed')
plt.show()

```

Problem 3

sensitivity analysis

```

import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# Constant setting
k = #0.02 evaporation coefficient, unit: m/s
V_max = #2 maximum volume, unit: m³
max_noise = #50 maximum noise, unit: dB
position_factor = 1.2# location factor, assuming the air purifier is located in the
center of the room

# Hypothetical function of power consumption and noise
def power_consumption(flow_speed):
    """ "Calculate power consumption based on airflow velocity (assuming
    proportional to the cube of airflow velocity)" """
    return 0.5 * flow_speed **3+50# power consumption is proportional to the cube
of airflow speed

```



```

def noise_level(flow_speed):
    """ "Calculate noise based on airflow velocity (assuming linear relationship with
    airflow velocity)" """
    return 40+2 * flow_speed# assumes a linear relationship between noise and
    airflow velocity

# Objective function of humidification efficiency
def humidification_efficiency(params, max_power):
    """
    Humidification efficiency is calculated.
    params[0] = parameter 1 (the size of the corresponding shape, such as radius,
    length, etc.)
    params[1] = parameter 2 (corresponding to airflow velocity)
    params[2] = parameter 3 (corresponding to humidification time)
    """
    shape_type = int(params[0]) # shape type (0 = cube, 1= column, 2 = box)
    size_params = params[1:4] # The parameters of the corresponding shape, such
    as the length, width, and height of the cuboid
    v = params[1] # Airflow velocity
    t = params[2] # humidification time

    # Calculation of water surface area
    if shape_type == 0: # cube
        A = size_params[0] **2# water surface area is the square of the side length
    elif shape_type == 1: # Cylindrical
        r = size_params[0] # radius
        A = np.pi * r **2# water surface area is sphere area
    else: # cuboid
        A = size_params[0] * size_params[1] # The water surface area is length
        multiplied by width

    Calculate humidification efficiency: efficiency = k * A * v * t * position factor
    efficiency = k * A * v * t * position_factor

    # Calculate power and noise
    power = power_consumption(v)
    noise = noise_level(v)

    # Penalty if power or noise exceeds limits
    if power > max_power:
        efficiency -= (power-max_power) * 10# Penalty: the part that exceeds the power
        limit

```

```
if noise > max_noise:
    efficiency -= (noise-max_noise) * 5# Penalty: Exceeding the noise limit

return -efficiency# for maximum humidification efficiency

# Set different maximum power values for sensitivity analysis
max_powers = np.linspace(100, 200, 100)
optimal_efficiencies = []

# Fixed parameters (here use the parameters corresponding to the results of the
previous optimization as an example)
fixed_params = [0, 1.25992105, 2.0, 24.0]

for max_power in max_powers:
    result = humidification_efficiency(fixed_params, max_power)
    optimal_efficiencies.append(-result)

# Visualize sensitivity analysis results
plt.figure(figsize=(8, 6))
plt.plot(max_powers, optimal_efficiencies)
plt.xlabel('maximum power (W)')
plt.ylabel('Maximum humidification efficiency')
plt.title('Sensitivity analysis of maximum power to maximum humidification
efficiency')
plt.show()
```

Problem 4

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Poly3DCollection
from scipy.optimize import minimize

plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False

# Room parameter setting
room_length = 8.0# room length (m)
room_width = 5.0# room width (m)
```

```
room_height = 3.0# room height (m)

# Device parameter setting
max_device_width = 2.5# maximum width (m)
max_device_height = 3# maximum height (m)
max_device_depth = 2.5# maximum depth (m)
max_device_radius = 2# max radius (m)

# Initial environmental parameters
initial_temp = 35.0# initial temperature (degree Celsius)
initial_humidity = 40.0# initial humidity (%)
initial_pollution = 80.0# initial pollutant concentration (unit)

# Target environment parameters
target_temp = 26.0# target temperature (deg C)
target_humidity = 50.0# target humidity (%)
target_pollution = 20.0# target pollutant concentration (unit)

# Simulated environment improvement
def simulate_environment(shape, size):
    if shape == 'rectangle':
        width, height, depth = size
        influence_area = 2 * (width * height + width * depth + height * depth)
    elif shape == 'cylinder':
        radius, height = size
        influence_area = 2 * np.pi * radius * height + 2 * np.pi * radius**2
    else:
        raise ValueError("Invalid shape")

    temp_improvement = influence_area * 0.1
    humidity_improvement = influence_area * 0.05
    pollution_reduction = influence_area * 0.2

    new_temp = initial_temp - temp_improvement
    new_humidity = initial_humidity + humidity_improvement
    new_pollution = initial_pollution - pollution_reduction

    new_temp = max(min(new_temp, initial_temp), target_temp)
    new_humidity = max(min(new_humidity, 100), target_humidity)
    new_pollution = max(new_pollution, target_pollution)

    return new_temp, new_humidity, new_pollution
```

```
# Objective function
def objective(params):
    shape_code = int(round(params[0]))
    size1 = params[1]
    size2 = params[2]
    size3 = params[3]

    penalty = 0
    if shape_code == 0:
        shape = 'rectangle'
        size = (size1, size2, size3)
        if size1 > max_device_width or size2 > max_device_height or size3 >
            max_device_depth:
            penalty += 1e6
        if size1 * size2 * size3 > 2:
            penalty += 1e6
    elif shape_code == 1:
        shape = 'cylinder'
        size = (size1, size3)
        if size1 > max_device_radius or size2 > max_device_height:
            penalty += 1e6
        if np.pi * size1**2 * size3 > 2:
            penalty += 1e6
        else:
            return 1e6

    new_temp, new_humidity, new_pollution = simulate_environment(shape, size)
    environment_improvement = (initial_temp-new_temp) + (new_humidity-
        initial_humidity) + (initial_pollution-new_pollution)

    total_objective = - environment_improvement + penalty
    return total_objective

# Weight setting
comfort_weight = 0.4
purification_weight = 0.4
efficiency_weight = 0.2

# Initial guess
initial_guess = [0, 0.5, 0.5, 0.5] # [shape_code, size1, size2, size3]
```

```
# optimization
result = minimize(objective, initial_guess, bounds=[(0, 1), (0.1,
max_device_width), (0.1, max_device_height), (0.1, max_device_depth)],
method='SLSQP')

# Output results
if result.success:
    optimal_params = result.x
    shape_code = int(round(optimal_params[0]))
    size1 = optimal_params[1]
    size2 = optimal_params[2]
    size3 = optimal_params[3]

    if shape_code == 0:
        shape = 'rectangle'
        size = (size1, size2, size3)
        print("Best 3-in -1 Device Design (Rectangular) :")
        print(f "AC: width = {size1:.2f}m, depth = {size3:.2f}m")
        print(f "Air Purifier: Width = {size1:.2f}m, Depth = {size3:.2f}m")
        print(f "Humidifier: width = {size1:.2f}m, depth = {size3:.2f}m")
    else:
        shape = 'cylinder'
        size = (size1, size3)
        print("Best 3-in -1 Device Design (Cylinder) :")
        print(f "AC: Radius = {size1:.2f}m, Height = {size3:.2f}m")
        print(f "Air Purifier: Radius = {size1:.2f}m, Height = {size3:.2f}m")
        print(f "Humidifier: Radius = {size1:.2f}m, Height = {size3:.2f}m")

# Draw the best equipment shape and mark the location of air conditioners, air
purifiers and humidifiers
def plot_device(shape, size):
    fig = plt.figure(figsize=(10, 8))
    ax = fig.add_subplot(111, projection='3d')

    if shape == 'rectangle ':
        width, height, depth = size
        x_center, y_center, z_center = room_length / 2, room_width / 2, room_height /
        2
        x0 = x_center-width/2
        y0 = y_center-height/2
        z0 = z_center-depth/2
```

```

corners = np.array ([
[x0, y0, z0],
[x0 + width, y0, z0]
[x0 + width, y0 + height, z0]
[x0, y0 + height, z0]
[x0, y0, z0 + depth]
[x0 + width, y0, z0 + depth]
[x0 + width, y0 + height, z0 + depth]
[x0, y0 + height, z0 + depth]
])

faces = [
[corners[0], corners[1], corners[5], corners[4]]
[corners[1], corners[2], corners[6], corners[5]]
[corners[2], corners[3], corners[7], corners[6]]
[corners[3], corners[0], corners[4], corners[7]]
[corners[4], corners[5], corners[6], corners[7]]
[corners[0], corners[1], corners[2], corners[3]]
]

poly3d = [faces[ I] for I in range(6)]
ax.add_collection3d(Poly3DCollection(poly3d, facecolors='mistyrose',
linewidths=1, edgecolors='black', alpha=0.6))

# Mark the location of air conditioners, air purifiers and humidifiers
# ax.text(x_center, y_center, z0 + depth + 0.1, f'Depth = {depth:.2f} m',
color='black', ha='center')
# ax.text(x_center, y0 - 0.1, z_center, f'Height = {height:.2f} m', color='black',
ha='center')
#ax.text(x0 - 0.1, y_center, z_center, f'Width = {width:.2f} m', color='black',
va='center', rotation=90)

# Device Module Location
ax.scatter(x_center, y_center, z_center + depth / 2, color='blue', s=100,
label='air conditioner')
ax.scatter(x_center, y_center, z_center - depth / 2, color='green', s=100,
label='air cleaner')
ax.scatter(x_center - width / 4, y_center, z_center, color='red', s=100,
label='humidifier')
print ('Maximum height of air conditioner: ')
print(z_center + height / 2)
print ('Maximum humidifier height: ')
print(z_center)
print ('Maximum air purifier height: ')

```

```
print(z_center - height / 2)

elif shape == 'cylinder ':
    radius, height = size
    z_center = room_height / 2
    theta = np.linspace(0, 2 * np.pi, 100)
    z = np.linspace(0, height, 100)
    Z, Theta = np.meshgrid(z, theta)
    X = radius * np.cos(Theta)
    Y = radius * np.sin(Theta)
    ax.plot_surface(X, Y, Z, color='mistyrose', edgecolor='black')

    ax.text(z_center, 0, height + 0.1, f'Height = {height:.2f} m', color='black',
            ha='center')
    ax.text(radius + 0.1, 0, z_center, f'Radius = {radius:.2f} m', color='black',
            va='center', rotation=90)

    # Device module location
    ax.scatter(0,0, z_center + height / 2, color = 'blue', s = 100, label = 'aircon ')
    ax.scatter(0,0, z_center-height/2, color = 'green', s = 100, label = 'air purier')
    ax.scatter(radius/2,0, z_center, color = 'red', s = 100, label = 'humidifier ')

    ax.set_xlabel('X')
    ax.set_ylabel('Y')
    ax.set_zlabel('Z')
    ax.set_title(f'Best 3-in-1 device ({shape})')
    plt.legend()
    plt.show()

    plot_device(shape, size)

else:
    print (" optimization failed! ")
    print(result.message)
```

